

Appunti di Metodi Numerici

Carbone Davide ¹

A.A. 2026/2027

¹Lezioni del prof. A. Borio

Indice

1	Aritmetica di Macchina	2
1.1	Rappresentazione dei numeri (reali)	2
1.2	Arrotondamento di numeri reali a numeri di macchina	3
1.3	Operazioni di macchina ed equivalenze	5
1.4	Cancellazione numerica	7
1.5	Condizionamento di un problema numerico	9
1.5.1	Cancellazione numerica come problema mal condizionato	10
1.6	Stabilità di un algoritmo	10
2	Sistemi Lineari	13
2.1	Norme	13
2.2	Forme speciali di matrici	15
2.3	Matrici a blocchi	18
2.4	Condizionamento di un sistema lineare	19
2.5	Metodi diretti	22
2.6	Metodi iterativi	28
2.6.1	Metodo di Jacobi e Gauss-Seidel	30
2.6.2	Generalizzazione di Jacobi e Gauss-Seidel	33
2.6.3	Convergenza di metetodi iterativi	34
2.6.4	Rilassamento del metodo di Gauss-Seidel	35
2.6.5	Metodi di discesa	37

Capitolo 1

Aritmetica di Macchina

Uno dei più grandi errori che si possono commettere quando si programma al calcolatore è quello di pensare che i risultati numerici ottenuti siano privi di errore. Per motivare questa affermazione definiremo nel seguito cosa si intende per un'aritmetica di macchina e le problematiche ad essa collegate.

1.1 Rappresentazione dei numeri (reali)

Definizione 1.1.1 (Rappresentazione floating-point). Si definisce rappresentazione floating-point in base N di un numero reale a la seguente espressione:

$$a = (-1)^s p N^q, \quad s \in \{0, 1\}, \quad p \geq 0 \in \mathbb{R}, \quad q \in \mathbb{Z}$$

Osservazione: Nonostante spesso ragioneremo e faremo esempi in base $N = 10$ le architetture dei computer classici usano $N = 2$.

Osservazione: La rappresentazione di un numero floating point non è unico. Per convenzione però si usa la rappresentazione seguente.

Definizione 1.1.2 (Rappresentazione floating-point **normalizzata**). Una rappresentazione floating-point si dice **normalizzata** se p soddisfa:

$$N^{-1} \leq p < 1$$

In questo caso p (mantissa) e q (esponente o caratteristica) sono univocamente determinati.

Una volta fissata una base N del sistema di numerazione, s , p e q quindi definiscono in modo univoco un numero reale a .

Un computer reale può memorizzare solo un numero finito di cifre per la mantissa e per l'esponente. Non tutti i numeri reali sono allora rappresentabili con notazione floating-point.

Definizione 1.1.3 (Numeri di macchina). Si definiscono **numeri di macchina** i numeri con mantissa ed esponente rappresentabili in un calcolatore.

L'insieme dei numeri di macchina in un calcolatore che memorizza t cifre per la mantissa e q

compreso tra $L < 0$ e $U > 0$ risulta così definito:

$$\mathbb{F} = \{0\} \cap \{(-1)^s 0.a_1 a_2 a_3 \dots a_t \cdot N^q, 0 \leq a_i < N, a_1 \neq 0, L \leq q \leq U\}$$

Definizione 1.1.4 (Aritmetica di macchina). Un'aritmetica di macchina è l'insieme delle regole e operazioni matematiche tra numeri di macchina rappresentati in una certa notazione floating-point normalizzata.

Osservazione: Il più piccolo numero rappresentabile è $m = 0.1000 \dots 0 \cdot N^L$.

Definizione 1.1.5. Si definisce **regione di underflow** l'insieme dei numeri reali diversi da 0 appartenenti a $(-m, m)$.

Osservazione: Il più grande numero rappresentabile è $M = 0.N - 1N - 1N - 1 \dots N - 1 \cdot N^U$.

Definizione 1.1.6. Si definisce **regione di overflow** l'insieme dei numeri reali appartenenti a $(-\inf, -M) \cup (M, +\inf)$.

Osservazione: È importante notare che anche se i dati iniziali e il risultato finale sono numeri di macchina o arrotondabili a numeri di macchina le operazioni intermedie potrebbero dare origine a numeri che appartengono alla regione di overflow (valore restituito 0) o di underflow (valore restituito $\inf$).

1.2 Arrotondamento di numeri reali a numeri di macchina

Se un numero reale possiede una mantissa normalizzata con più di t cifre il calcolatore ricorre ad una tecnica di arrotondamento chiamata "rounding to even".

Definizione 1.2.1 (Tecnica "rounding to even"). La mantissa p viene approssimata con la mantissa di macchina $\bar{p}$ più vicina e se p è equidistante da due mantisse di macchina consecutive allora p viene approssimata con quella delle due che ha l'ultima cifra pari.

Più formalmente se

$$p = a_1 a_2 \dots a_t a_{t+1} \dots a_n$$

e

$$p \in (\bar{p}_1, \bar{p}_2)$$

con

$$\bar{p}_1 = 0.a_1 a_2 \dots a_t \quad e \quad \bar{p}_2 = \bar{p}_1 + N^{-t}$$

due mantisse di macchina consecutive, allora la mantissa di macchina $\bar{p}$ corrispondente a p è data da

$$\bar{p} = \begin{cases} \bar{p}_1, & \text{se } p \in (\bar{p}_1, \frac{\bar{p}_1 + \bar{p}_2}{2}) \text{ oppure } p = \frac{\bar{p}_1 + \bar{p}_2}{2} \text{ e } a_t \text{ è pari} \\ \bar{p}_2, & \text{se } p \in (\frac{\bar{p}_1 + \bar{p}_2}{2}, \bar{p}_2) \text{ oppure } p = \frac{\bar{p}_1 + \bar{p}_2}{2} \text{ e } a_t \text{ è dispari} \end{cases}$$

Osservazione: Si introduce la tecnica perché statisticamente arrotondare sempre per eccesso porta ad un errore maggiore. Intuitivamente infatti l'errore non viene compensato da altri calcoli. **Aggiungi in che senso questo errore ha media nulla dopo aver fatto statistica**

Nel seguito $\bar{a}$ sarà il numero di macchina associato al numero reale a . È evidente che ogni volta che si memorizza $\bar{a}$ al posto di a si commette un errore.

Definizione 1.2.2 (Errore). Sia $\tilde{x}$ un'approssimazione del numero x . Si definisce **errore assoluto** associato a $\tilde{x}$

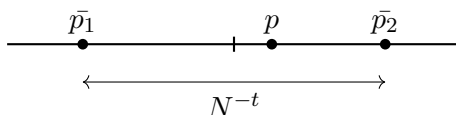
$$e_a = |x - \tilde{x}|$$

e **errore relativo**

$$e_r = \frac{|x - \tilde{x}|}{|x|}, \quad x \neq 0$$

Definizione 1.2.3 (Errore di arrotondamento). Si definisce **errore di arrotondamento** l'errore che si commette quando si sostituisce un numero reale a con un numero di macchina $\bar{a}$.

Determiniamo ora una stima dell'errore assoluto $|a - \bar{a}|$ e dell'errore relativo $\frac{|a - \bar{a}|}{|a|}$ di arrotondamento. Sia $a = (-1)^s p N^q$ con $N^{-1} \leq p < 1$ e $\bar{a} = (-1)^s \bar{p} N^q$. Supponiamo che $p \in (\bar{p}_1, \bar{p}_2)$, con $\bar{p}_1, \bar{p}_2$ mantisse di macchina consecutive.



Si ha allora necessariamente che

$$|p - \bar{p}| \leq \frac{1}{2} N^{-t}$$

L'errore assoluto risulta quindi

$$|a - \bar{a}| = |(-1)^s p N^q - (-1)^s \bar{p} N^q| = |p - \bar{p}| N^q \leq \frac{1}{2} N^{q-t}$$

e dal momento che $p \geq N^{-1}$

$$\frac{|a - \bar{a}|}{|a|} = \frac{|p - \bar{p}|}{|p|} \leq \frac{1}{2} N^{-t} \frac{1}{|p|} \leq \frac{1}{2} N^{1-t}$$

Osservazione: La distanza tra due numeri di macchina consecutivi è

$$|\bar{a}_1 - \bar{a}_2| = |(-1)^s \bar{p}_1 N^q - (-1)^s \bar{p}_2 N^q| = |\bar{p}_1 - \bar{p}_2| N^q = N^{q-t}$$

Osserviamo quindi che i numeri di macchina non sono equamente distribuiti in quanto si allargano all'aumentare di q .

Definizione 1.2.4. Si definisce **epsilon di macchina** la quantità

$$eps = N^{1-t}$$

Si definisce **precisione di macchina** la quantità

$$\varepsilon_m = \frac{1}{2}N^{1-t}$$

Osservazione: L'epsilon di macchina è il massimo errore relativo tra due numeri di macchina consecutivi.

La precisione di macchina è una costante specifica di ogni aritmetica floating-point, d'altronde dipende dalla base numerica del calcolatore (N) e dal numero di cifre con cui è rappresentata la mantissa (t).

Definendo $\varepsilon = \frac{(a-\bar{a})}{a}$ otteniamo

$$\bar{a} = a(1 + \varepsilon), \quad |\varepsilon| \leq \varepsilon_m$$

Osservazione: Matlab usa un'aritmetica di macchina con $N = 2$, la tecnica di arrotondamento "rounded to even" e 64 bit per la rappresentazione del numero di cui

1. 1 bit per il segno
2. 52 bit per la mantissa p
3. 11 bit per l'esponente q

Inoltre Matlab mette a disposizione i seguenti valori

1. `realmin` fornisce il più piccolo numero di macchina (modulo)
2. `realmax` fornisce il più grande numero di macchina (modulo)
3. `eps` fornisce il valore dell'epsilon di macchina

Osservazione: Nei calcolatori reali i bit per la mantissa sono 53 dal momento che in base 2 la prima cifra dopo la virgola deve necessariamente essere 0.

Osservazione: Nei computer moderni la precisione di macchina è dell'ordine di 10^{-16} . Da questo fatto equivale a dire che possiamo ritenere affidabile un risultato ottenuto in modo tradizionale solo fino alla 16-esima cifra significativa.

1.3 Operazioni di macchina ed equivalenze

Tralasciando rare eccezioni il risultato di un'operazione tra due numeri di macchina **non** è un numero di macchina.

Un calcolatore infatti non può eseguire esattamente le operazioni aritmetiche esatte ma può solamente effettuare le **operazioni di macchina**.

Definizione 1.3.1. Un'operazione di macchina $\odot$ associa a 2 numeri di macchina un terzo numero di macchina, ottenuto arrotondando il risultato della operazione in aritmetica esatta $\cdot$:

$$\bar{a}_1 \odot \bar{a}_2 = \overline{\bar{a}_1 \cdot \bar{a}_2} = (\bar{a}_1 \cdot \bar{a}_2)(1 + \varepsilon_\odot), \quad |\varepsilon_\odot| \leq \varepsilon_m.$$

Definizione 1.3.1. Un'operazione di macchina $\odot$ associa a 2 numeri di macchina un terzo numero di macchina, ottenuto arrotondando il risultato della operazione in aritmetica esatta $\cdot$:

$$\bar{a}_1 \odot \bar{a}_2 = \overline{\bar{a}_1 \cdot \bar{a}_2} = (\bar{a}_1 \cdot \bar{a}_2)(1 + \varepsilon_\odot), \quad |\varepsilon_\odot| \leq \varepsilon_m.$$

Osservazione: $|\varepsilon_{\odot}|$ è l'errore relativo dell'operazione $\odot$ infatti

$$\bar{a}_1 \odot \bar{a}_2 = (\bar{a}_1 \cdot \bar{a}_2) + \varepsilon_{\odot}(\bar{a}_1 \cdot \bar{a}_2) \Leftrightarrow \varepsilon_{\odot} = \frac{\bar{a}_1 \odot \bar{a}_2 - \bar{a}_1 \cdot \bar{a}_2}{\bar{a}_1 \cdot \bar{a}_2}$$

Le operazioni $+, -, \times, /$ sono rappresentate dalle corrispondenti operazioni di macchina $\oplus, \ominus, \otimes, \oslash$.

Per le operazioni di macchina rimane valida la proprietà commutativa:

$$\bar{a}_1 \oplus \bar{a}_2 = \bar{a}_2 \oplus \bar{a}_1$$

$$\bar{a}_1 \otimes \bar{a}_2 = \bar{a}_2 \otimes \bar{a}_1$$

ma non rimangono valide la proprietà associativa e distributiva:

$$\bar{a}_1 \oplus (\bar{a}_2 \oplus \bar{a}_3) \neq (\bar{a}_1 \oplus \bar{a}_2) \oplus \bar{a}_3$$

$$a_1 \otimes (a_2 \otimes a_3) \neq (a_1 \otimes a_2) \otimes a_3$$

$$\bar{a}_1 \otimes (\bar{a}_2 \oplus \bar{a}_3 \neq (\bar{a}_1 \otimes \bar{a}_2) \oplus (\bar{a}_1 \otimes \bar{a}_3))$$

$$(\bar{a}_1 \otimes \bar{a}_2) \oslash \bar{a}_3 \neq (\bar{a}_1 \oslash \bar{a}_3) \otimes \bar{a}_2$$

Un altro fenomeno di cui bisogna essere a conoscenza è

$\bar{a}_1 \oplus \bar{a}_2 = \bar{a}_1, \quad |\bar{a}_2| < |\bar{a}_1|eps$

quando viene effettuata questa somma infatti le t cifre di mantissa vengono occupate prima con le cifre di $\bar{a}_1$ e poi con alcuni zeri, dal momento che le cifre significative di $\bar{a}_2$ dovrebbero essere collocate dopo la t -esima cifra della nuova mantissa.

cifre mantissa di $\bar{a}_1$ cifre mantissa di $\bar{a}_2$

←→←→

0.13746234700000000000000000000000000000000000000387423847

←----->

t cifre

Esempio 1.3.2. Diamo un esempio pratico del fenomeno appena descritto. Vogliamo effettuare la somma

$$\sum_{i=1}^{10^{16}} 10^{-16} = 1$$

al calcolatore. Un'idea di implementazione in MATLAB potrebbe essere

1 $s=0$

```

2 for i=1:1:10^16
3     s=s+10^-16
4 end

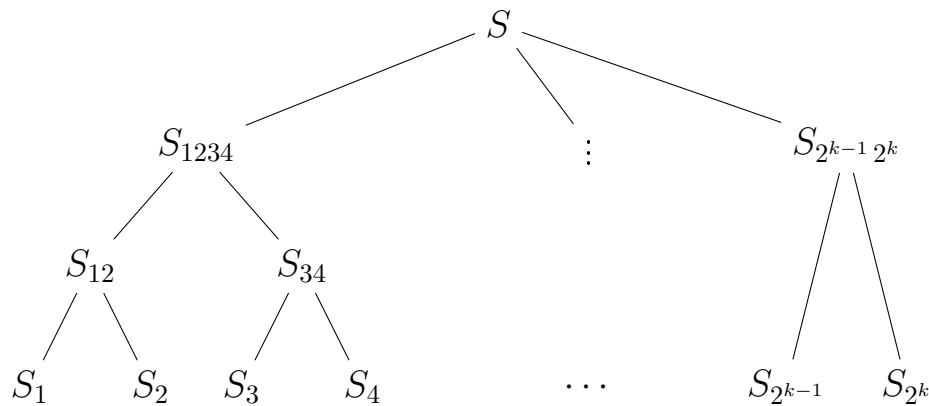
```

Quando $i = \frac{10^{16}}{2}$, $S = \frac{1}{2}$, allora per $i = \frac{10^{16}}{2} + 1$,

$$S = \frac{1}{2} + 10^{-16} = \frac{1}{2}.$$

E così via per ogni $i > \frac{10^{16}}{2} + 1$.

Un metodo per effettuare la somma evitando errori è quello di effettuare un algoritmo su un albero binario del tipo



Raggruppando i numeri da sommare in gruppi S_i e sommando in parallelo dal fondo del grafo fino ad ottenere S .

Definizione 1.3.3 (Concetto di equivalenza). Due espressioni $e_1(x)$ ed $e_2(x)$ si definiscono equivalenti nell'aritmetica del calcolatore quando i loro valori in aritmetica di macchina ottenuti tramite le approssimazioni in aritmetica di macchina dei dati $\bar{e}_1(\bar{x})$, $\bar{e}_2(\bar{x})$ differiscono di una tolleranza relativa dello stesso ordine di grandezza o inferiore della precisione di macchina ε_m , ovvero

$$\frac{|\bar{e}_1 - \bar{e}_2|}{|\bar{e}_1|} \lesssim \varepsilon_m$$

1.4 Cancellazione numerica

Definizione 1.4.1 (Cancellazione numerica). Siano $\bar{a}_1 = (-1)^{s_1} \bar{p}_1 N^{q_1}$, $\bar{a}_2 = (-1)^{s_2} \bar{p}_2 N^{q_2}$ le rappresentazione di macchina dei due numeri reali $a_1 = (-1)^{s_1} p_1 N^{q_1}$, $a_2 = (-1)^{s_2} p_2 N^{q_2}$.

Il fenomeno della **cancellazione numerica** consiste nella perdita di cifre della mantissa quando si effettua l'operazione di sottrazioni tra due numeri di macchina e sussistono le seguenti condizioni:

1. $s_1 = s_2$
2. $q_1 = q_2$

$$3. \bar{p}_1 \approx \bar{p}_2$$

$$4. \bar{p}_1 \neq p_1 \text{ e/o } \bar{p}_2 \neq p_2$$

Questo fenomeno si presenta perché in un'aritmetica finita, come abbiamo visto in precedenza, le operazioni vengono effettuate su numeri di macchina. Se uno o più numeri di macchina coinvolti presentano una perturbazione rispetto ai dati esatti l'operazione di macchina potrebbe ampliarla. Per capire meglio il fenomeno consideriamo il seguente esempio.

Esempio 1.4.2. Consideriamo i numeri $a_1 = 0.157824831$ e $a_2 = 0.157348212$.

In aritmetica esatta $a_1 - a_2 = 0.000476619 = 0.476619 \cdot 10^{-3}$.

Siano $\bar{a}_1 = 0.15782$ e $\bar{a}_2 = 0.15735$ le due rappresentazioni di a_1 e a_2 in un'aritmetica finita con $N = 10$ e $t = 5$.

In questa aritmetica $\bar{a}_1 \ominus \bar{a}_2 = 0.00047 = 0.47000 \cdot 10^{-3}$.

Come possiamo notare abbiamo perso 3 cifre dal risultato in aritmetica esatta.

Calcoliamo gli errori relativi su $\bar{a}_1$, $\bar{a}_2$ e $(\bar{a}_1 \ominus \bar{a}_2)$:

$$\frac{|a_1 - \bar{a}_1|}{|a_1|} = 0.3 \cdot 10^{-4} < \varepsilon_m$$

$$\frac{|a_2 - \bar{a}_2|}{|a_2|} = 0.1 \cdot 10^{-4} < \varepsilon_m$$

$$\frac{|(a_1 - a_2) - (\bar{a}_1 \ominus \bar{a}_2)|}{|a_1 - a_2|} = 0.1 \cdot 10^{-1} > \varepsilon_m$$

Nonostante gli errori relativi su $\bar{a}_2$ e $\bar{a}_1$ siano minori della precisione di macchina (e quindi $a_1 \sim \bar{a}_1$ e $a_2 \sim \bar{a}_2$) l'errore relativo della loro differenza calcolata in aritmetica di macchina è maggiore della precisione di macchina e quindi le differenze non sono equivalenti. Notiamo che in questo caso

$$\bar{a}_1 \ominus \bar{a}_2 = \bar{a}_1 - \bar{a}_2$$

e quindi la cancellazione prescinde dal fatto che si usino operazioni di macchina o esatte ma dipende principalmente dal fatto che gli operandi sono perturbati rispetto ai dati esatti. Questo fatto sarà più chiaro nel seguito dopo aver introdotto il condizionamento di un problema. L'operazione di macchina può poi introdurre ulteriore errore ma questo prescinde dal fenomeno della cancellazione numerica.

A volte è possibile manipolare l'espressione algebrica che identifica un problema numerico per evitare la cancellazione numerica.

Esempio 1.4.3. Consideriamo il problema numerico $y = \sqrt{x + \delta} - \sqrt{x}$.

Se $|\delta| \ll x$ allora il problema di cancellazione numerica può essere eliminato razionalizzando:

$$y = (\sqrt{x + \delta} - \sqrt{x}) \frac{\sqrt{x + \delta} + \sqrt{x}}{\sqrt{x + \delta} + \sqrt{x}} = \frac{\delta}{\sqrt{x + \delta} + \sqrt{x}}$$

Se invece $\delta \approx -x$ il fenomeno della cancellazione numerica non può essere evitato.

Un altro modo per manipolare un'espressione algebrica del tipo $y = f(x)$ per evitare il fenomeno della cancellazione numerica è quello di sostituire ad $f(x)$ il suo sviluppo di Taylor centrato in un valore prossimo a quello di x .

Esempio 1.4.4. Consideriamo il problema numerico:

$$y = \frac{e^x - 1}{x}$$

Per $x \approx 0$ possiamo scrivere il problema come:

$$y = \frac{1}{x} \left(1 + x + \frac{x^2}{2!} + \dots + \frac{x^n}{n!} + o(x^n) \right) + 1 = 1 + \frac{x}{2!} + \dots + \frac{x^{n-1}}{n!} + o(x^{n-1})$$

Ricordando il fenomeno per cui $\bar{a}_1 \oplus \bar{a}_2 = \bar{a}_1$, $|\bar{a}_2| < |\bar{a}_1|eps$, possiamo limitarci a sommare i soli termini che danno un contributo reale in aritmetica finita.

Nello studio della propagazione degli errori bisogna distinguere il ruolo del problema dal ruolo dell'algoritmo utilizzato per risolverlo.

Nelle successive due sezioni daremo le definizioni di **condizionamento di un problema numerico** e di **stabilità di un algoritmo** e spiegheremo la loro relazione.

1.5 Condizionamento di un problema numerico

Definizione 1.5.1. Si definisce **problema numerico** una relazione funzionale $y = f(x)$ (**esplicita**) oppure $f(x, y) = 0$ (**implicita**), tra dati x (**input**) e risultati y (**output**).

Qui x e y sono simboli generici: non indicano necessariamente scalari.

I dati x e i risultati y devono essere rappresentabili tramite numeri, vettori o matrici di dimensione finita. Nel caso di vettori o matrici l'errore assoluto o relativo associato a essi viene definito tramite una norma di vettore oppure di matrice.

Definizione 1.5.2 (Condizionamento di un problema numerico). Sia $y = f(x)$ un generico problema numerico e siano:

1. $\bar{x}$ una perturbazione dei dati x di input
2. $\bar{y}$ i risultati ottenuti dai dati $\bar{x}$ in aritmetica esatta

Un problema si dice **ben condizionato** se accade che l'errore relativo associato a $\bar{x}$ è dello stesso ordine o maggiore a quello associato a $\bar{y}$ ovvero se

$$\frac{\|y - \bar{y}\|}{\|y\|} \lesssim \frac{\|x - \bar{x}\|}{\|x\|}$$

In caso contrario si dice **mal condizionato**.

Possiamo studiare il condizionamento di un problema con stime del tipo:

$$\frac{\|y - \bar{y}\|}{\|y\|} \leq K(f, x) \frac{\|x - \bar{x}\|}{\|x\|}$$

Definizione 1.5.3. Si definisce **numero di condizionamento** del problema il più piccolo valore di $K(f, x)$ per il quale la relazione precedente risulta verificata. Se K è abbastanza piccolo il problema risulta ben condizionato

Quindi in un problema ben condizionato a perturbazioni nei dati corrispondono perturbazioni nei risultati dello stesso ordine di grandezza. In un problema mal condizionato invece a piccole perturbazioni nei dati corrispondono grandi perturbazioni nei risultati.

1.5.1 Cancellazione numerica come problema mal condizionato

Inquadriamo la cancellazione numerica come problema mal condizionato. In particolare calcoliamo il numero di condizionamento della somma. Dati $a_1, a_2 \in \mathbb{R}$ siano $\bar{a}_1, \bar{a}_2 \in \mathbb{R}$ le loro rappresentazioni di macchina. Poniamo $S = a_1 + a_2$ e $\bar{S} = \bar{a}_1 + \bar{a}_2$. Vogliamo trovare $K(a_1, a_2)$ tale che

$$\frac{|S - \bar{S}|}{|S|} \leq K(a_1, a_2) \cdot \frac{\|\mathbf{x} - \bar{\mathbf{x}}\|}{\|\mathbf{x}\|}$$

per qualche norma p e con $\mathbf{x} = (a_1, a_2)$ e $\bar{\mathbf{x}} = (\bar{a}_1, \bar{a}_2)$. Partiamo dall'errore relativo

$$|S - \bar{S}| = |a_1 + a_2 - \bar{a}_1 - \bar{a}_2| \leq |a_1 - \bar{a}_1| + |a_2 - \bar{a}_2|$$

Allora

$$\frac{|S - \bar{S}|}{|S|} \leq \frac{|a_1 - \bar{a}_1| + |a_2 - \bar{a}_2|}{|a_1 + a_2|} = \frac{|a_1| + |a_2|}{|a_1 + a_2|} \cdot \frac{|a_1 - \bar{a}_1| + |a_2 - \bar{a}_2|}{|a_1| + |a_2|} = \frac{|a_1| + |a_2|}{|a_1 + a_2|} \cdot \frac{\|\mathbf{x} - \bar{\mathbf{x}}\|_1}{\|\mathbf{x}\|_1}$$

Avendo posto $\mathbf{x} = (a_1, a_2)$ e $\bar{\mathbf{x}} = (\bar{a}_1, \bar{a}_2)$. Se chiamiamo

$$K(a_1, a_2) = \frac{|a_1| + |a_2|}{|a_1 + a_2|}$$

troviamo che

$$a_1 \rightarrow -a_2 \Rightarrow K(a_1, a_2) \rightarrow +\infty$$

e questo è esattamente il caso della cancellazione numerica.

Abbiamo quindi dato un fondamento teorico al problema: l'operazione di somma sotto quelle ipotesi è mal condizionata ovvero la perturbazione dei dati di input viene amplificata nell'output. Nell'aritmetica del calcolatore quella perturbazione è data dall'approssimazione di macchina dei dati in input.

1.6 Stabilità di un algoritmo

Come osservavamo nel paragrafo nello scorso paragrafo il condizionamento di un problema dipende dal fatto che i dati siano perturbati e non dal fatto di eseguire operazioni di macchina. Questo ci porta ad osservare che il condizionamento di un problema prescinde dall'algoritmo risolutivo utilizzato per risolverlo. Abbiamo anche detto però che le operazioni di macchina possono introdurre ulteriore errore. Per quantificare questo errore si introduce il concetto di stabilità di un algoritmo di risoluzione.

Definizione 1.6.1. Sia $y = f(x)$ un problema numerico generico. Data un'aritmetica finita siano inoltre:

1. $\bar{x}$ la rappresentazione di macchina dei dati x
2. $\bar{y}$ i risultati ottenuti attraverso un algoritmo a partire da $\bar{x}$ in aritmetica esatta
3. $\tilde{y}$ i risultati ottenuti attraverso un algoritmo a partire da $\bar{x}$ in aritmetica finita

L'algoritmo si dice **numericamente stabile** se accade che l'errore relativo

$$\frac{\|\bar{y} - \tilde{y}\|}{\|\bar{y}\|} \lesssim \varepsilon_m$$

In caso contrario si dice **instabile**.

Osservazione: Un algoritmo è quindi numericamente stabile quando la sequenza delle operazioni non amplifica eccessivamente gli errori di arrotondamento presente nei dati.

Esempio 1.6.2. Consideriamo l'algoritmo per il calcolo di π definito dalla successione

$$x_1 = 2,$$

$$x_n = 2^{n-\frac{1}{2}} \sqrt{1 - \sqrt{1 - 4^{1-n} x_{n-1}^2}}, \quad n \geq 2.$$

Questo algoritmo è instabile a causa del fenomeno della cancellazione numerica. Infatti, al crescere di n , la quantità

$$z = \sqrt{1 - 4^{1-n} x_{n-1}^2}$$

si avvicina sempre di più a 1. Di conseguenza, nell'operazione

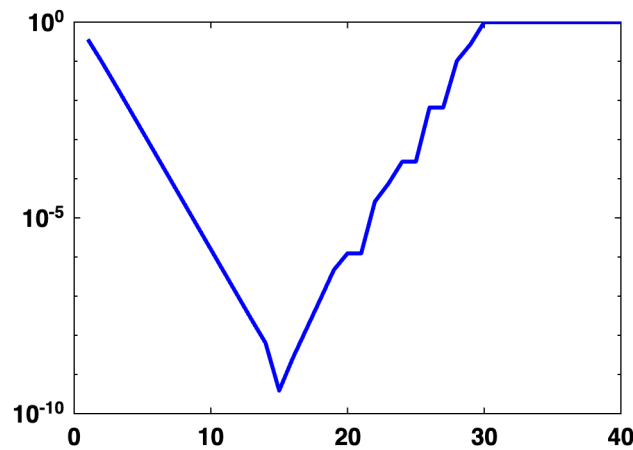
$$1 - z$$

si effettua la sottrazione tra due numeri sempre più vicini, con conseguente perdita di cifre significative.

L'errore relativo

$$\frac{|\pi - x_n|}{|\pi|}$$

inizialmente diminuisce, ma successivamente aumenta proprio a causa dell'instabilità numerica dell'algoritmo.



È possibile eliminare la cancellazione numerica razionalizzando l'espressione precedente. In questo modo la stessa successione può essere calcolata mediante

$$x_1 = 2,$$

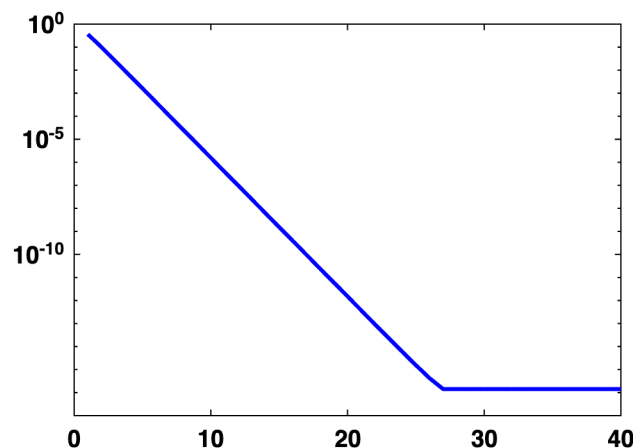
$$x_n = x_{n-1} \sqrt{\frac{2}{1 + \sqrt{1 - 4^{1-n} x_{n-1}^2}}}, \quad n \geq 2.$$

In questa forma non compare più la sottrazione tra quantità molto vicine e l'algoritmo risulta stabile.

In questo caso l'errore relativo

$$\frac{|\pi - x_n|}{|\pi|}$$

decresce fino a raggiungere un valore dell'ordine della precisione di macchina, oltre il quale non può diminuire ulteriormente.



Capitolo 2

Sistemi Lineari

Il fine di questo capitolo è quello di presentare al lettore i primi metodi numerici per la risoluzione di sistemi lineari. Prima di addentrarci nella trattazione dei metodi verranno presentate alcune nozioni riguardanti vettori e matrici.

2.1 Norme

Definizione 2.1.1 (Norma di un vettore). Sia $\mathbf{v} \in \mathbb{R}^n$ un vettore.

Si definisce norma di un vettore una funzione $f : \mathbb{R}^n \rightarrow \mathbb{R}$ che rispetta le seguenti caratteristiche:

1. $f(\mathbf{v}) \geq 0 \quad \forall \mathbf{v} \in \mathbb{R}^n$ e $f(\mathbf{v}) = 0 \Leftrightarrow \mathbf{v} = \mathbf{0}$
2. $f(\alpha \mathbf{v}) = \alpha f(\mathbf{v}) \quad \forall \mathbf{v} \in \mathbb{R}^n, \quad \forall \alpha \in \mathbb{R}$
3. $f(\mathbf{w} + \mathbf{v}) \leq f(\mathbf{v}) + f(\mathbf{w})$

Nel seguito verranno considerate i seguenti tipi di norma di un vettore:

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$$

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\mathbf{x}^T \mathbf{x}} \quad \text{Norma euclidea}$$

$$\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|$$

MatLab fornisce funzioni per il calcolo delle norme:

```
1 norm(x,1); % Fornisce la norma 1 del vettore x
2 norm(x,2); % Fornisce la norma 2 del vettore x
3 norm(x); % Fornisce la norma 2 del vettore x
4 norm(x, inf); % Fornisce la norma infinita del vettore x
```

Osservazione: Su V spazio vettoriale di dimensione finita tutte le norme sono equivalenti ovvero fissate $\|\cdot\|_\star, \|\cdot\|_0 \exists C_\star, C^\star > 0$ tali che

$$C_\star \|\mathbf{v}\|_0 \leq \|\mathbf{v}\|_\star \leq C^\star \|\mathbf{v}\|_0 \quad \forall \mathbf{v} \in V$$

Il concetto di norma può essere esteso anche ad un operatore lineare in dimensione finita (e quindi ad una matrice).

Definizione 2.1.2. Sia $L_{\mathbf{A}} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, $\mathbf{x} \mapsto \mathbf{Ax}$, $\mathbf{A} \in \mathbb{R}^{m,n}$ e sia $\|\cdot\|_*$ una norma di vettore. Si definisce norma operatoriale associata a $\|\cdot\|_*$ la norma

$$\|L_{\mathbf{A}}\|_* = \max_{\mathbf{x} \in \mathbb{R}^n} \frac{\|\mathbf{Ax}\|_*}{\|\mathbf{x}\|_*} = \max_{\substack{\mathbf{x} \in \mathbb{R}^n \\ \|\mathbf{x}\|_* = 1}} \|\mathbf{Ax}\|_*$$

Equivalentemente indichiamo con $\|\mathbf{A}\|_*$ la norma di matrice definita da $\|\mathbf{A}\|_* = \|L_{\mathbf{A}}\|_*$.

Definizione 2.1.3. Sia $\mathbf{B} \in \mathbb{R}^{m,m}$ di autovalori $\{\lambda_i\}_{i=1}^m$. Definiamo raggio spettrale di $\mathbf{B}$ la quantità

$$\rho(\mathbf{B}) = \max_{1 \leq i \leq m} |\lambda_i|$$

In particolare siamo interessati alle norme

•

$$\|\mathbf{A}\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|$$

•

$$\|\mathbf{A}\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|$$

•

$$\|\mathbf{A}\|_2 = \sqrt{\rho(\mathbf{A}^T \mathbf{A})} \quad (\text{norma spettrale})$$

Le norme appena definite si ottengono come norme operatoriali associate alle norme vettoriali $p = 1, 2, \infty$.

Osservazione: Notiamo che la norma 1 prende il valore massimo tra le norme dei vettori colonna, mentre la norma infinita prende il valore massimo tra le norme dei vettori righe. Risulta quindi valida la seguente relazione:

$$\|\mathbf{A}\|_\infty = \|\mathbf{A}^T\|_1$$

Segnaliamo l'esistenza della norma 2 di una matrice $\|\mathbf{A}\|_2$, detta norma spettrale, che verrà trattata in seguito.

```
1 norm(A,1); % Fornisce la norma 1 della matrice A
2 norm(A,inf); % Fornisce la norma infinito della matrice A
3 norm(A,2); % Fornisce la norma spettrale della matrice A
```

Definizione 2.1.4 (Compatibilità della norma di matrice con la norma di vettore). Una norma di matrice $\|\mathbf{A}\|_p$ si dice compatibile con una norma di vettore $\|\mathbf{x}\|_p$ quando sussiste la seguente relazione:

$$\|\mathbf{A}\mathbf{x}\|_p \leq \|\mathbf{A}\|_p \cdot \|\mathbf{x}\|_p$$

In generale tale relazione è verificata per $p = \{1, 2, \infty\}$. Inoltre per ogni norma di matrice compatibile con una norma di vettore vale la seguente relazione

$$\rho(\mathbf{A}) \leq \|\mathbf{A}\|$$

2.2 Forme speciali di matrici

In questo paragrafo elencheremo una serie di forme "speciali" che possono avere le matrici.

Definizione 2.2.1 (Triangolare superiore). Una matrice $\mathbf{A} \in \mathbb{R}^{m,n}$ si definisce **triangolare superiore** se $a_{ij} = 0, \quad i > j$

$$\mathbf{A} = \begin{pmatrix} * & * & * \\ 0 & * & * \\ 0 & 0 & * \end{pmatrix}$$

Definizione 2.2.2 (Triangolare inferiore). Una matrice $\mathbf{A} \in \mathbb{R}^{m,n}$ si definisce **triangolare inferiore** se $a_{ij} = 0, \quad i < j$

$$\mathbf{A} = \begin{pmatrix} * & 0 & 0 \\ * & * & 0 \\ * & * & * \end{pmatrix}$$

Definizione 2.2.3 (Diagonale). Una matrice $\mathbf{A} \in \mathbb{R}^{m,n}$ si definisce **diagonale** se $a_{ij} = 0, \quad i \neq j$

$$\mathbf{A} = \begin{pmatrix} * & 0 & 0 \\ 0 & * & 0 \\ 0 & 0 & * \end{pmatrix}$$

Definizione 2.2.4 (Tridiagonale). Una matrice $\mathbf{A} \in \mathbb{R}^{m,n}$ si definisce **tridiagonale** se $a_{ij} = 0, \quad |i - j| > 1$

$$\mathbf{A} = \begin{pmatrix} * & * & 0 \\ * & * & * \\ 0 & * & * \end{pmatrix}$$

Definizione 2.2.5 (Trasposta). Si definisce **trasposta** della matrice $\mathbf{A}$ la matrice $\mathbf{A}^T$ le cui entrate sono definite come $a_{ij}^T = a_{ji}$

Definizione 2.2.6 (Simmetrica). Una matrice $\mathbf{A}$ si dice **simmetrica** se $\mathbf{A} = \mathbf{A}^T$

Definizione 2.2.7 (Inversa). Si definisce **inversa** della matrice $\mathbf{A} \in \mathbb{R}^{n,n}$ la matrice $\mathbf{A}^{-1}$ tale che $\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$. Non sempre si può definire l'inversa.

Definizione 2.2.8 (Ortogonale). Una matrice $\mathbf{A} \in \mathbb{R}^{n,n}$ si dice **ortogonale** se $\mathbf{A}\mathbf{A}^T = \mathbf{A}^T\mathbf{A} = \mathbf{I}$. Pertanto l'inversa di una matrice ortogonale è la sua trasposta.

Osservazione: Le colonne di una matrice ortogonale formano una base ortonormale di $\mathbb{R}^n$ rispetto al prodotto scalare standard.

Definizione 2.2.9 (Matrice di permutazione). Si definisce **matrice di permutazione** $\mathbf{P}$ una matrice ottenuta permutando le righe della matrice identità. Moltiplicando a destra o a sinistra una matrice o un vettore per una matrice di permutazione si applica la permutazione alla matrice o al vettore. In particolare

- $\mathbf{P}\mathbf{A}$ scambia le righe di $\mathbf{A}$
- $\mathbf{A}\mathbf{P}$ scambia le colonne di $\mathbf{A}$
- $\mathbf{P}\mathbf{v}$ scambia le entrate (righe) di $\mathbf{v}$
- $\mathbf{v}^T\mathbf{P}$ scambia le entrate (colonne) di $\mathbf{v}^T$

Osservazione: Le matrici di permutazioni sono ortogonali.

Esempio 2.2.10. Consideriamo la seguente matrice di permutazione

$$\mathbf{P} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

calcoliamo i prodotti $\mathbf{P}\mathbf{A}$ e $\mathbf{P}\mathbf{b}$, con $\mathbf{A} \in \mathbb{R}^{3,3}$ e $\mathbf{b} \in \mathbb{R}^3$

$$\mathbf{P}\mathbf{A} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} = \begin{pmatrix} a_{31} & a_{32} & a_{33} \\ a_{21} & a_{22} & a_{23} \\ a_{11} & a_{12} & a_{13} \end{pmatrix}$$

$$\mathbf{P}\mathbf{b} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} = \begin{pmatrix} b_3 \\ b_2 \\ b_1 \end{pmatrix}$$

Definizione 2.2.11 (Diagonale dominante per righe). Si dice **diagonale dominante per righe** una matrice le cui entrate rispettano la seguente proprietà

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| \quad \text{per } i = 1, \dots, n$$

Ovvero se per ogni riga l'elemento sulla diagonale principale è in modulo maggiore della somma dei moduli degli altri elementi della riga

Definizione 2.2.12 (Diagonale dominante per colonne). Si dice **diagonale dominante per colonne** una matrice le cui entrate rispettano la seguente proprietà

$$|a_{jj}| > \sum_{\substack{i=1 \\ i \neq j}}^n |a_{ij}| \quad \text{per } j = 1, \dots, n$$

Ovvero se per ogni colonna l'elemento sulla diagonale principale è in modulo maggiore della somma dei moduli degli altri elementi della colonna

Definizione 2.2.13. Una matrice simmetrica $\mathbf{A}$ si dice definita positiva se

$$\mathbf{x}^T \mathbf{A} \mathbf{x} > 0 \quad \text{per ogni } \mathbf{x} \neq \mathbf{0};$$

oppure, equivalentemente, se

$$\lambda_i(\mathbf{A}) > 0 \quad \text{per ogni } i = 1, 2, \dots, n$$

con $\lambda_i(\mathbf{A})$ autovalore di $\mathbf{A}$; oppure, equivalentemente, se

$$\det(\mathbf{A}_k) > 0 \quad \text{per ogni } k = 1, 2, \dots, n$$

ove $\mathbf{A}_k$ è la matrice di ordine k formata dall'intersezione delle prime k righe e k colonne di $\mathbf{A}$.

Teorema 2.2.14. La matrice $\mathbf{A} = \mathbf{B}^T \mathbf{B}$ con $\mathbf{B} \in \mathbb{R}^{n,n}$ avente determinate non nullo è definita positiva.

Dimostrazione. $\mathbf{A}$ è simmetrica per costruzione, infatti

$$\mathbf{A} = \mathbf{B}^T \mathbf{B} \quad \mathbf{A}^T = (\mathbf{B}^T \mathbf{B})^T = \mathbf{B}^T (\mathbf{B}^T)^T = \mathbf{A}$$

Sia $\mathbf{x} \neq \mathbf{0}$ e $\mathbf{y} = \mathbf{B}\mathbf{x}$

$$\mathbf{x}^T \mathbf{A} \mathbf{x} = \mathbf{x}^T (\mathbf{B}^T \mathbf{B}) \mathbf{x} = \mathbf{x}^T \mathbf{B}^T \mathbf{B} \mathbf{x} = (\mathbf{x}^T \mathbf{B}^T) (\mathbf{B} \mathbf{x}) = (\mathbf{B} \mathbf{x})^T (\mathbf{B} \mathbf{x}) = \mathbf{y}^T \mathbf{y}$$

$$\mathbf{y}^T \mathbf{y} = [y_1, \dots, y_n] \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} = y_1^2 + y_2^2 + \dots + y_n^2 = \|\mathbf{y}\|_2^2$$

$$\Rightarrow \mathbf{x}^T \mathbf{A} \mathbf{x} = \|\mathbf{y}\|_2^2 \geq 0 \quad \text{e} \quad \|\mathbf{y}\|_2^2 = 0 \Leftrightarrow \mathbf{y} = \mathbf{0}$$

Dal momento che $\mathbf{y} \neq \mathbf{0}$ perché, essendo $\mathbf{y} = \mathbf{B}\mathbf{x}$, $\mathbf{x} \neq \mathbf{0}$ e $\mathbf{B}$ di rango massimo, si può concludere che

$$\mathbf{x}^T \mathbf{A} \mathbf{x} > 0$$

□

2.3 Matrici a blocchi

Definizione 2.3.1 (Matrice a blocchi). Una **matrice a blocchi** è una matrice descritta in termini delle sue sottomatrici anziché in termini dei suoi elementi. Una descrizione di una matrice a blocchi non è unica.

$$\mathbf{A} = \left(\begin{array}{cc|c} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ \hline a_{31} & a_{32} & a_{33} \end{array} \right) = \left(\begin{array}{cc} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{array} \right)$$

La **trasposta** di una **matrice a blocchi** si ottiene nel seguente modo:

$$\mathbf{A}^T = \left(\begin{array}{cccc} \mathbf{A}_{11} & \mathbf{A}_{12} & \cdots & \mathbf{A}_{1n} \\ \mathbf{A}_{21} & \mathbf{A}_{22} & \cdots & \mathbf{A}_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}_{m1} & \mathbf{A}_{m2} & \cdots & \mathbf{A}_{mn} \end{array} \right)^T = \left(\begin{array}{cccc} \mathbf{A}_{11}^T & \mathbf{A}_{21}^T & \cdots & \mathbf{A}_{m1}^T \\ \mathbf{A}_{12}^T & \mathbf{A}_{22}^T & \cdots & \mathbf{A}_{m2}^T \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}_{1n}^T & \mathbf{A}_{2n}^T & \cdots & \mathbf{A}_{mn}^T \end{array} \right)$$

Quando il numero dei blocchi e le loro dimensioni sono compatibili, le operazioni tra matrici a blocchi possono essere effettuate trattando i blocchi come scalari.

Esempio 2.3.2. Siano

$$\mathbf{A} = \left(\begin{array}{cc} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{array} \right), \quad \mathbf{B} = \left(\begin{array}{cc} \mathbf{B}_{11} & \mathbf{B}_{12} \\ \mathbf{B}_{21} & \mathbf{B}_{22} \end{array} \right)$$

due matrici con la stessa partizione a blocchi.

Si ha allora

$$\mathbf{A} + \mathbf{B} = \left(\begin{array}{cc} \mathbf{A}_{11} + \mathbf{B}_{11} & \mathbf{A}_{12} + \mathbf{B}_{12} \\ \mathbf{A}_{21} + \mathbf{B}_{21} & \mathbf{A}_{22} + \mathbf{B}_{22} \end{array} \right), \quad \alpha \mathbf{A} = \left(\begin{array}{cc} \alpha \mathbf{A}_{11} & \alpha \mathbf{A}_{12} \\ \alpha \mathbf{A}_{21} & \alpha \mathbf{A}_{22} \end{array} \right)$$

$$\mathbf{AB} = \left(\begin{array}{cc} \mathbf{A}_{11}\mathbf{B}_{11} + \mathbf{A}_{12}\mathbf{B}_{21} & \mathbf{A}_{11}\mathbf{B}_{12} + \mathbf{A}_{12}\mathbf{B}_{22} \\ \mathbf{A}_{21}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{21} & \mathbf{A}_{21}\mathbf{B}_{12} + \mathbf{A}_{22}\mathbf{B}_{22} \end{array} \right)$$

Quest'ultimo risultato è alla base degli algoritmi di calcolo matriciale dei calcolatori dal momento che riduce il calcolo di un prodotto di matrici a somme di prodotti di matrici più piccoli. È particolarmente utile nel caso di matrici sparse (che presentano larghe aree formate da 0), perché si individuano dei blocchi formati interamente da zeri che non inficiano nel calcolo

Definizione 2.3.3 (Matrice triangolare a blocchi). Una matrice triangolare superiore (inferiore / superiore) a blocchi è una matrice quadrata che ha blocchi quadrati sulla diagonale e i cui blocchi sotto

(sopra / sotto) la diagonale principale contengono solo zeri:

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} & \cdots & \cdots & \mathbf{A}_{1n} \\ \mathbf{0} & \mathbf{A}_{22} & \cdots & \cdots & \mathbf{A}_{2n} \\ \vdots & \vdots & \ddots & & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & & \mathbf{A}_{nn} \end{pmatrix}$$

Per le matrici diagonali a blocchi vale il seguente risultato:

$$\det(\mathbf{A}) = \prod_{i=1}^n \det(\mathbf{A}_{ii})$$

2.4 Condizionamento di un sistema lineare

Consideriamo il sistema lineare di n equazioni nelle n incognite $x_1, \dots, x_n$:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \quad \quad \quad \ddots \quad \quad \quad \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

Il sistema può essere riscritto in forma matriciale e si ha:

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} \iff \mathbf{A}\mathbf{x} = \mathbf{b}$$

Assumiamo che il sistema abbia un' unica soluzione, in questo caso la matrice $\mathbf{A}$ si dice **non singolare** ($\det(\mathbf{A}) \neq 0$), dal momento che ha rango massimo.

Andiamo a studiare il condizionamento del nostro problema numerico. Forniremo la dimostrazione del caso con $\mathbf{A}$ non perturbata, $\bar{\mathbf{b}}$ perturbato e $\bar{\mathbf{x}}$ soluzione in aritmetica esatta del sistema $\mathbf{A}\bar{\mathbf{x}} = \bar{\mathbf{b}}$. Successivamente enunceremo senza dimostrare il caso più generale con $\bar{\mathbf{A}}$ e $\bar{\mathbf{b}}$ perturbati e $\bar{\mathbf{x}}$ soluzione in aritmetica esatta del sistema $\bar{\mathbf{A}}\bar{\mathbf{x}} = \bar{\mathbf{b}}$.

Teorema 2.4.1. Dato un problema numerico del tipo $\mathbf{A}\bar{\mathbf{x}} = \bar{\mathbf{b}}$ vale la seguente relazione

$$\frac{\|\mathbf{x} - \bar{\mathbf{x}}\|_p}{\|\mathbf{x}\|_p} \leq K_p(\mathbf{A}) \frac{\|\mathbf{b} - \bar{\mathbf{b}}\|_p}{\|\mathbf{b}\|_p}$$

con $K_p(\mathbf{A}) = \|\mathbf{A}\|_p \|\mathbf{A}^{-1}\|_p$

Dimostrazione. Consideriamo i due problemi numerici

$$\mathbf{A}\bar{\mathbf{x}} = \bar{\mathbf{b}} \quad \mathbf{A}\mathbf{x} = \mathbf{b}$$

Se $\mathbf{b} = \mathbf{0}$ allora $\bar{\mathbf{b}} = \mathbf{0}$ e l'unica soluzione del sistema lineare è $\mathbf{x} = \bar{\mathbf{x}} = \mathbf{0}$. Se $\mathbf{b} \neq \mathbf{0}$ sottraendo membro a membro otteniamo

$$\mathbf{A}(\bar{\mathbf{x}} - \mathbf{x}) = \bar{\mathbf{b}} - \mathbf{b} \Rightarrow \bar{\mathbf{x}} - \mathbf{x} = \mathbf{A}^{-1}(\bar{\mathbf{b}} - \mathbf{b}) \Rightarrow$$

$$\Rightarrow \|\bar{x} - x\|_p = \|\mathbf{A}^{-1}(\bar{\mathbf{b}} - \mathbf{b})\|_p \leq \|\mathbf{A}^{-1}\|_p \|\bar{\mathbf{b}} - \mathbf{b}\|_p$$

$$\mathbf{A}x = \mathbf{b} \Rightarrow \|\mathbf{b}\|_p = \|\mathbf{A}x\|_p \leq \|x\|_p \|\mathbf{A}\|_p \Rightarrow \|x\|_p \geq \frac{\|\mathbf{b}\|_p}{\|\mathbf{A}\|_p} \Rightarrow$$

$$\Rightarrow \frac{1}{\|x\|_p} \leq \frac{\|\mathbf{A}\|_p}{\|\mathbf{b}\|_p}$$

$$\frac{\|\bar{x} - x\|_p}{\|x\|_p} \leq \|\mathbf{A}^{-1}\|_p \|\mathbf{A}\|_p \frac{\|\bar{\mathbf{b}} - \mathbf{b}\|_p}{\|\mathbf{b}\|_p}$$

$$K_p(\mathbf{A}) = \|\mathbf{A}^{-1}\|_p \|\mathbf{A}\|_p$$

□

Teorema 2.4.2. Dato un problema numerico del tipo $\bar{\mathbf{A}}\bar{x} = \bar{\mathbf{b}}$.

Se $\|\mathbf{A} - \bar{\mathbf{A}}\|_p < \frac{1}{2\|\mathbf{A}^{-1}\|_p}$ vale la seguente relazione

$$\frac{\|x - \bar{x}\|_p}{\|x\|_p} \leq 2K_p(\mathbf{A}) \left(\frac{\|\mathbf{A} - \bar{\mathbf{A}}\|_p}{\|\mathbf{A}\|_p} + \frac{\|\mathbf{b} - \bar{\mathbf{b}}\|_p}{\|\mathbf{b}\|_p} \right)$$

con

$$K_p(\mathbf{A}) = \|\mathbf{A}\|_p \|\mathbf{A}^{-1}\|_p$$

$K_p(\mathbf{A})$ viene definito **numero di condizionamento** del sistema $\mathbf{A}x = \mathbf{b}$.

Osservazione: Per compatibilità si ha che

$$\|\mathbf{A}\mathbf{B}x\|_p \leq \|\mathbf{A}\|_p \cdot \|\mathbf{B}x\|_p \leq \|\mathbf{A}\|_p \cdot \|\mathbf{B}\|_p \cdot \|x\|_p \Rightarrow \frac{\|\mathbf{A}\mathbf{B}x\|_p}{\|x\|_p} \leq \|\mathbf{A}\|_p \cdot \|\mathbf{B}\|_p$$

Allora

$$\|\mathbf{A}\mathbf{B}\|_p = \max_{x \in \mathbb{R}^n} \frac{\|\mathbf{A}\mathbf{B}x\|_p}{\|x\|_p} \leq \|\mathbf{A}\|_p \cdot \|\mathbf{B}\|_p$$

Osservazione: Si osservi che

$$K_p(\mathbf{A}) = \|\mathbf{A}\|_p \|\mathbf{A}^{-1}\|_p \geq \|\mathbf{A}\mathbf{A}^{-1}\|_p = \|\mathbf{I}\|_p = \max_{x \in \mathbb{R}^n} \frac{\|\mathbf{I}x\|_p}{\|x\|_p} = \max_{x \in \mathbb{R}^n} \frac{\|x\|_p}{\|x\|_p} = 1$$

e quindi che

1. se $K(\mathbf{A}) \approx 1$, la matrice $\mathbf{A}$ si dice **ben condizionata** $\Rightarrow$ a piccole perturbazioni sui dati corrispondono perturbazioni sui risultati al più dello stesso ordine di grandezza
2. se $K(\mathbf{A}) \gg 1$ la matrice $\mathbf{A}$ si dice **mal condizionata** $\Rightarrow$ a piccole perturbazioni sui dati possono corrispondere grandi perturbazioni sui risultati

Esempio 2.4.3.

$$\mathbf{D} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 10^{-10} \end{bmatrix}$$

$$K_1(\mathbf{D}) = \|\mathbf{D}\|_1 \|\mathbf{D}^{-1}\|_1 = 10^{10}.$$

$\mathbf{D}$ è mal condizionata.

Osservazione: Denotato con $\lambda_i(\mathbf{B})$ il generico autovalore della matrice $\mathbf{B}$, si dimostra che:

$$K_2(\mathbf{A}) = \frac{\sqrt{\max_i |\lambda_i(\mathbf{A}^T \mathbf{A})|}}{\sqrt{\min_i |\lambda_i(\mathbf{A}^T \mathbf{A})|}}$$

Se $\mathbf{A}$ è simmetrica, allora

$$K_2(\mathbf{A}) = \frac{|\max_i \lambda_i(\mathbf{A})|}{|\min_i \lambda_i(\mathbf{A})|}$$

Se $\mathbf{A}$ è simmetrica e definita positiva, allora

$$K_2(\mathbf{A}) = \frac{\max_i \lambda_i(\mathbf{A})}{\min_i \lambda_i(\mathbf{A})}$$

Nel caso simmetrico dire che l'autovalore minimo e massimo hanno moduli di ordine di grandezza diverso vuol dire che $\mathbf{A}$ trasforma lo spazio in modo non uniforme. È proprio questa proprietà che amplifica gli errori di perturbazione degli input.

Due classiche matrici mal condizionate sono la **matrice di Hilbert**

$$\mathbf{H}_n = \begin{pmatrix} 1 & \frac{1}{2} & \cdots & \frac{1}{n} \\ \frac{1}{2} & \frac{1}{3} & \cdots & \frac{1}{n+1} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1}{n} & \frac{1}{n+1} & \cdots & \frac{1}{2n-1} \end{pmatrix}$$

e la **matrice di Vandermonde**

$$\mathbf{V}_n = \begin{pmatrix} x_1^n & x_1^{n-1} & \cdots & x_1 & 1 \\ x_2^n & x_2^{n-1} & \cdots & x_2 & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{n+1}^n & x_{n+1}^{n-1} & \cdots & x_{n+1} & 1 \end{pmatrix}$$

.

Il condizionamento di un sistema lineare **non** dipende dalla **norma scelta**. Infatti tutte le norme hanno lo stesso ordine di grandezza.

Seguono una serie di funzioni MatLab utili per il calcolo del condizionamento di un sistema lineare e per generare le matrici di Hilbert e di Vandermonde

```

1 cond(A,1) % Calcola il numero di condizionamento in norma 1 del sistema Ax
  = b
2 cond(A, inf) % Calcola il numero di condizionamento in norma infinito del
  sistema Ax = b
3 hilb(n) % genera la matrice di Hilbert di ordine n
4 vander(x) % Genera la matrice di Vandermonde associata al vettore x con
  componenti x(1) , ... , x(n+1)

```

2.5 Metodi diretti

In questa e nella prossima sezione presenteremo diversi metodi numerici per la risoluzione di sistemi lineari non singolari. Dividiamo i metodi in due grandi classi: i metodi diretti e quelli iterativi. Dei metodi diretti daremo solo un breve accenno dal momento che questi sono stati già trattati in modo esaustivo nel corso del primo anno. Le principali proprietà dei metodi diretti per la risoluzione del sistema lineare

$$\mathbf{A}\mathbf{x} = \mathbf{b}$$

sono:

1. $\mathbf{x}$ viene determinata mediante un numero finito di passi (al più $n - 1$ se n è l'ordine del sistema);
2. $\mathbf{x}$ in aritmetica con precisione infinita di calcolo viene determinata in maniera esatta; in aritmetica con precisione finita di calcolo viene determinata con una precisione la cui entità non dipende dalle richieste dell'utente;
3. i metodi diretti modificano la matrice dei coefficienti $\mathbf{A}$ e il termine noto $\mathbf{b}$;
4. i metodi diretti sono efficienti per matrici dense e di piccole-medie dimensioni.

Osservazione: A volte modificano la quantità di memoria allocata.

La scelta di un metodo diretto ed efficiente per la risoluzione del sistema lineare

$$\mathbf{A}\mathbf{x} = \mathbf{b}$$

dipende dalla struttura e dalle proprietà della matrice $\mathbf{A}$.

Matrici diagonali

Se la matrice dei coefficienti $\mathbf{A}$ è **diagonale**, con $a_{kk} \neq 0$, $k = 1, \dots, n$, la soluzione del sistema lineare

$$\begin{cases} a_{11}x_1 = b_1, \\ a_{22}x_2 = b_2, \\ a_{33}x_3 = b_3, \\ \vdots \\ a_{nn}x_n = b_n \end{cases}$$

si ricava immediatamente mediante le seguenti n divisioni:

```
1 for k = 1:n
2     x(k) = b(k) / A(k,k);
3 end
```

Osservazione: L'algoritmo per i sistemi diagonali essendo formato da un solo ciclo **for** ha costo computazionale $O(n)$.

Matrici trinagolari

Se la matrice dei coefficienti $\mathbf{A}$ è **triangolare superiore**, con $a_{kk} \neq 0$, $k = 1, \dots, n$, la soluzione del sistema lineare

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1, \\ \vdots \\ a_{n-1,n-1}x_{n-1} + a_{n-1,n}x_n = b_{n-1}, \\ a_{nn}x_n = b_n \end{cases}$$

si ottiene ricavando l'incognita x_n dall'ultima equazione, x_{n-1} dalla penultima equazione (dopo aver sostituito in essa il valore di x_n), e continuando verso l'alto fino a ricavare x_1 dalla prima (dopo aver sostituito in essa i valori di $x_2, \dots, x_n$).

Lo schema di calcolo è il seguente:

```

1 x(n) = b(n) / A(n,n);
2
3 for k = n-1:-1:1
4     x(k) = (b(k) - sum(A(k,k+1:n) .* x(k+1:n))) / A(k,k);
5 end

```

Tale procedura è nota con il nome di **metodo di sostituzione all'indietro**. Il costo computazionale in termini di operazioni aritmetiche è essenzialmente $O(n^2/2)$ per n grande.

Se la matrice dei coefficienti $\mathbf{A}$ è **triangolare inferiore**, con $a_{kk} \neq 0$, $k = 1, \dots, n$, la soluzione del sistema lineare

$$\begin{cases} a_{11}x_1 = b_1, \\ a_{21}x_1 + a_{22}x_2 = b_2, \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

si ottiene in modo simile al caso triangolare superiore ma partendo dalla prima equazione e arrivando fino all'ultima ovvero attraverso lo schema di calcolo

```

1 x(1) = b(1) / A(1,1);
2
3 for k = 2:n
4     x(k) = (b(k) - sum(A(k,1:k-1) .* x(1:k-1))) / A(k,k);
5 end

```

Tale procedura è nota con il nome di **metodo di sostituzione in avanti**. Anche in questo caso il costo computazionale in termini di operazioni aritmetiche è essenzialmente $O(n^2/2)$ per n grande.

Osservazione: Gli elementi sono in tutto

$$1 + 2 + \cdots + n = \sum_{i=1}^n i = \frac{n(n+1)}{2}.$$

Per ogni elemento si fa un numero finito di operazioni quindi ho costo

$$O\left(\alpha \frac{n(n+1)}{2}\right) = O\left(\frac{n(n+1)}{2}\right) = O\left(\frac{n^2}{2}\right).$$

In particolare ad ogni iterazione faccio

- 1 sottrazione
- un numero di moltiplicazioni e somme pari al numero di coefficienti sulla riga k

In tutto $(n-1)$ sottrazioni e

$$\frac{n(n+1)}{2}$$

moltiplicazioni.

Metodo di Gauss

Se la matrice dei coefficienti $\mathbf{A}$ non ha una struttura particolare, il metodo diretto più noto e più utilizzato è senza dubbio il metodo delle eliminazioni di Gauss.

Si ricorda che il metodo delle eliminazioni di Gauss consta essenzialmente di due fasi:

1. trasformazione, in $n-1$ passi, del sistema assegnato $\mathbf{Ax} = \mathbf{b}$ nel sistema $\mathbf{Ux} = \bar{\mathbf{b}}$ equivalente a quello assegnato (ovvero che ammette la stessa soluzione $\mathbf{x}$), con $\mathbf{U}$ matrice triangolare superiore;
2. risoluzione del sistema $\mathbf{Ux} = \bar{\mathbf{b}}$ mediante la tecnica di sostituzione all'indietro.

Il costo computazionale del metodo delle eliminazioni di Gauss in termini di operazioni aritmetiche è essenzialmente $O(n^3/3)$ per n grande. Per garantire una **migliore stabilità numerica** dell'algoritmo di Gauss e, quindi, per ridurre l'amplificazione degli errori di arrotondamento che vengono generati nel corso delle trasformazioni, a ogni passo conviene operare un preliminare scambio di equazioni prima di procedere con le trasformazioni. Tale strategia è nota sotto il nome di **pivoting parziale** e consiste al passo k nello scambiare le righe in modo tale da avere a_{kk} massimo evitando così di dividere per un numero vicino allo zero.

Il pivoting è **superfluo** quando:

- $\mathbf{A}$ è a diagonale dominante per colonne;
- $\mathbf{A}$ è simmetrica e definita positiva.

A seguire il codice implementato senza pivoting parziale.

```
1 for k = 1:n-1 % selezione colonna
2     % Tecnica di pivoting parziale (codice omissso)
3     for i = k+1:n % fisso la riga
4         m(i,k) = A(i,k) / A(k,k);
5
6         for j = k:n % trasformo la riga
7             A(i,j) = A(i,j) - m(i,k) * A(k,j);
8         end
9     end
10 end
```

Osservazione: Il comando $\mathbf{A} \setminus \mathbf{b}$ in MATLAB calcola la soluzione $\mathbf{x}$ del sistema $\mathbf{Ax} = \mathbf{b}$ con il metodo più efficiente in base al tipo di matrice.

Fattorizzazione $\mathbf{PA} = \mathbf{LU}$

Il metodo delle eliminazioni di Gauss con pivoting parziale realizza la seguente fattorizzazione della matrice $\mathbf{A}$:

$$\mathbf{PA} = \mathbf{LU}$$

ove $\mathbf{U}$ è la matrice triangolare superiore ottenuta alla fine del metodo di Gauss, $\mathbf{L}$ è la matrice triangolare inferiore contenente gli $m(i, k)$ per $k = 1 : n - 1$ e $i = k + 1 : n$ avente diagonale unitaria e $\mathbf{P}$ è una matrice di permutazione, definita dagli scambi richiesti dalla strategia del pivoting. Il costo computazionale della fattorizzazione $\mathbf{PA} = \mathbf{LU}$ della matrice $\mathbf{A}$ è uguale a quello del metodo delle eliminazioni di Gauss e vale all'incirca $O(n^3/3)$ per n grande. In MATLAB

```
[L,U,P] = lu(A)
```

calcola i fattori $\mathbf{L}$, $\mathbf{U}$ e $\mathbf{P}$ della fattorizzazione $\mathbf{PA} = \mathbf{LU}$ di $\mathbf{A}$.

Si osservi che per la risoluzione di un sistema lineare $\mathbf{Ax} = \mathbf{b}$ si può procedere utilizzando la fattorizzazione $\mathbf{PA} = \mathbf{LU}$ nel seguente modo:

$$\mathbf{Ax} = \mathbf{b} \implies \mathbf{PAx} = \mathbf{Pb} \implies \mathbf{L} \underbrace{\mathbf{Ux}}_{\mathbf{y}} = \mathbf{Pb}$$

da cui segue la risoluzione dei due seguenti sistemi triangolari

$$\begin{cases} \mathbf{Ly} = \mathbf{Pb} & \implies \mathbf{y} \\ \mathbf{Ux} = \mathbf{y} & \implies \mathbf{x} \end{cases}$$

Se i fattori $\mathbf{L}$, $\mathbf{U}$, $\mathbf{P}$ della fattorizzazione $\mathbf{PA} = \mathbf{LU}$ non si devono calcolare perché già noti, il costo computazionale della risoluzione del sistema lineare $\mathbf{Ax} = \mathbf{b}$ è soltanto $O(n^2)$; altrimenti, il costo è pari a quello della fattorizzazione $\mathbf{PA} = \mathbf{LU}$, cioè $O(n^3/3)$ (essendo, per n grande, il costo della risoluzione dei due sistemi lineari triangolari trascurabile rispetto al costo della fattorizzazione).

Osservazione: Inoltre il metodo di eliminazione di Gauss è utile per calcolare il determinante di una matrice

$$\mathbf{PA} = \mathbf{LU} \implies \det(\mathbf{PA}) = \det(\mathbf{LU}) \implies \det(\mathbf{P}) \det(\mathbf{A}) = \det(\mathbf{L}) \det(\mathbf{U}).$$

$$\det(\mathbf{P}) = (-1)^S \quad S = \text{numero scambi.}$$

$$\det(\mathbf{L}) = 1.$$

$$\det(\mathbf{U}) = \prod_{i=1}^n u_{ii}.$$

$$\implies \det(\mathbf{A}) = (-1)^S \det(\mathbf{U}).$$

Fattorizzazione di Choleski

Per una matrice $\mathbf{A}$ simmetrica e definita positiva esiste ed è unica la fattorizzazione

$$\mathbf{A} = \mathbf{R}^T \mathbf{R}$$

ove $\mathbf{R}$ è una matrice triangolare superiore con elementi positivi sulla diagonale principale.

Tale fattorizzazione è detta fattorizzazione di Choleski e viene dimostrata nel seguente teorema.

Teorema 2.5.1. Se $\mathbf{A}$ è una matrice simmetrica e definita positiva allora esiste un'unica matrice triangolare inferiore $\mathbf{L}_1 (= \mathbf{R}^T)$ con elementi diagonali positivi tale che

$$\mathbf{A} = \mathbf{L}_1 \mathbf{L}_1^T.$$

Dimostrazione. È possibile dimostrare che se i determinanti delle matrici $\mathbf{A}_k$, formate dall'intersezione delle prime k righe e k colonne di una qualsiasi matrice $\mathbf{A}$, sono diversi da zero per $k = 1, \dots, n-1$, allora esiste ed è unica la fattorizzazione $\mathbf{LU}$ di $\mathbf{A}$.

(A meno di permutazioni)

Poiché per ipotesi $\mathbf{A}$ è simmetrica e definita positiva, e dunque verifica

$$\det(\mathbf{A}_k) > 0$$

per $k = 1, \dots, n$, allora possiamo senz'altro affermare che esiste un'unica matrice $\mathbf{L}$ triangolare inferiore con diagonale unitaria e un'unica matrice $\mathbf{U}$ triangolare superiore, tali che

$$\mathbf{A} = \mathbf{LU}.$$

Autovalori di $\mathbf{A}_k$ sottoinsieme di valori di $\mathbf{A}$.

Se denotiamo con $\mathbf{D}$ la matrice diagonale i cui elementi principali sono quelli di $\mathbf{U}$ e con $\mathbf{U}_1$ la matrice triangolare superiore con diagonale unitaria tale che $\mathbf{DU}_1 = \mathbf{U}$, allora possiamo scrivere

$$\mathbf{A} = \mathbf{LDU}_1.$$

Poiché $\mathbf{A} = \mathbf{LU}$ è simmetrica ed $\mathbf{L}$ è unica allora $\mathbf{U}_1 = \mathbf{L}^T$; infatti, da

$$\mathbf{A} = \mathbf{LDU}_1$$

segue

$$\mathbf{A} = \mathbf{A}^T = \mathbf{U}_1^T \mathbf{D} \mathbf{L}^T,$$

con $\mathbf{U}_1^T$ triangolare inferiore con diagonale unitaria. Per l'unicità di $\mathbf{L}$, si ha allora $\mathbf{L} = \mathbf{U}_1^T$ e $\mathbf{DU}_1 = \mathbf{U} = \mathbf{DL}^T$. Pertanto, risulta univocamente

$$\mathbf{A} = \mathbf{LDL}^T.$$

Inoltre, poiché $\mathbf{A}$ è definita positiva, gli elementi diagonali della matrice $\mathbf{D}$ sono tutti positivi in quanto anche $\mathbf{D}$ è definita positiva; infatti, per $\mathbf{x} \neq \mathbf{0}$, si ha

$$0 < \mathbf{x}^T \mathbf{A} \mathbf{x} = \mathbf{x}^T \mathbf{LDL}^T \mathbf{x} = (\mathbf{L}^T \mathbf{x})^T \mathbf{D} (\mathbf{L}^T \mathbf{x}) = \mathbf{y}^T \mathbf{D} \mathbf{y}$$

con $\mathbf{y} = \mathbf{L}^T \mathbf{x} \neq \mathbf{0}$, essendo $\mathbf{x} \neq \mathbf{0}$ e $\mathbf{L}$ non singolare.

Pertanto, denotando con $\mathbf{D}^{\frac{1}{2}}$ la matrice i cui elementi diagonali sono definiti come la radice quadrata degli elementi diagonali della matrice $\mathbf{D}$, allora

$$\mathbf{D} = \mathbf{D}^{\frac{1}{2}} \mathbf{D}^{\frac{1}{2}}$$

e

$$\mathbf{A} = \mathbf{L} \mathbf{D} \mathbf{L}^T = \mathbf{L} \mathbf{D}^{\frac{1}{2}} \mathbf{D}^{\frac{1}{2}} \mathbf{L}^T = \mathbf{L}_1 \mathbf{L}_1^T$$

avendo posto $\mathbf{L}_1 = \mathbf{L} \mathbf{D}^{\frac{1}{2}}$. Per costruzione $\mathbf{L}_1$ è triangolare inferiore con elementi diagonali positivi e, per l'unicità di $\mathbf{L}$ e di $\mathbf{D}$, $\mathbf{L}_1$ è unica. $\square$

Il calcolo del fattore di Choleski $\mathbf{R}$ si ottiene mediante un algoritmo il cui costo computazionale è essenzialmente $O(n^3/6)$ per n grande.

In MATLAB il comando

```
1 R = chol(A)
```

calcola il fattore $\mathbf{R}$ triangolare superiore della fattorizzazione di Choleski

$$\mathbf{A} = \mathbf{R}^T \mathbf{R},$$

della matrice simmetrica e definita positiva $\mathbf{A}$.

Per la risoluzione del sistema lineare $\mathbf{A} \mathbf{x} = \mathbf{b}$, con $\mathbf{A}$ simmetrica e definita positiva, nota la fattorizzazione di Choleski $\mathbf{A} = \mathbf{R}^T \mathbf{R}$, un algoritmo efficiente è il seguente:

$$\mathbf{A} \mathbf{x} = \mathbf{b} \implies \mathbf{R}^T \underbrace{\mathbf{R} \mathbf{x}}_{\mathbf{y}} = \mathbf{b} \implies \begin{cases} \mathbf{R}^T \mathbf{y} = \mathbf{b} & \implies \mathbf{y} \\ \mathbf{R} \mathbf{x} = \mathbf{y} & \implies \mathbf{x} \end{cases}$$

Fattorizzazione QR

Un'altra fattorizzazione per una generica matrice $\mathbf{A}$ di dimensioni $m \times n$ è la **fattorizzazione QR**. In particolare, si può dimostrare che esiste una matrice ortogonale $\mathbf{Q}$ di dimensioni $m \times m$ tale che

$$\mathbf{A} = \mathbf{Q} \mathbf{R}$$

dove gli elementi r_{ij} della matrice $\mathbf{R}$ sono nulli per $i > j$. Se $m = n$, $\mathbf{R}$ è una matrice triangolare superiore.

In MATLAB il comando

```
1 [Q, R] = qr(A)
```

calcola i fattori $\mathbf{Q}$ e $\mathbf{R}$ della fattorizzazione $\mathbf{A} = \mathbf{Q} \mathbf{R}$ di $\mathbf{A}$.

Questa fattorizzazione consente di costruire un algoritmo stabile per la risoluzione del sistema $\mathbf{A} \mathbf{x} = \mathbf{b}$, con $\mathbf{A}$ di ordine n e non singolare:

$$\mathbf{A} \mathbf{x} = \mathbf{b} \implies \mathbf{Q} \mathbf{R} \mathbf{x} = \mathbf{b} \implies \mathbf{R} \mathbf{x} = \mathbf{Q}^T \mathbf{b}$$

Tuttavia il calcolo delle matrici $\mathbf{Q}$ ed $\mathbf{R}$ richiede $O(2n^3/3)$ operazioni aritmetiche mentre il metodo di Gauss ne richiede solamente $O(n^3/3)$. Un'altra importante fattorizzazione di una generica matrice $\mathbf{A}$ di dimensioni $m \times n$ è la **decomposizione ai valori singolari (SVD)**. In particolare, si può dimostrare che esistono due matrici ortogonali $\mathbf{U}$ di dimensioni $m \times m$ e $\mathbf{V}$ di dimensioni $n \times n$ tali che

$$\mathbf{A} = \mathbf{U} \mathbf{S} \mathbf{V}^T$$

dove $\mathbf{S}$ è diagonale, ovvero

$$s_{ij} = \begin{cases} \sigma_i & \text{se } i = j, \\ 0 & \text{se } i \neq j \end{cases}$$

con

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0, \quad p = \min\{m, n\}.$$

Gli elementi diagonali della matrice $\mathbf{S}$ sono detti **valori singolari** della matrice $\mathbf{A}$. Essi coincidono con la radice quadrata degli autovalori (tutti non negativi) della matrice $\mathbf{A}^T \mathbf{A}$.

In MATLAB il comando

```
[U, S, V] = svd(A)
```

calcola i fattori $\mathbf{U}$, $\mathbf{S}$, e $\mathbf{V}$ della fattorizzazione $\mathbf{A} = \mathbf{USV}^T$ di $\mathbf{A}$.

Si può dimostrare che il rango della matrice $\mathbf{A}$ coincide con il numero dei valori singolari non nulli. Anche questa fattorizzazione consente di costruire un algoritmo stabile per la risoluzione del sistema $\mathbf{Ax} = \mathbf{b}$, con $\mathbf{A}$ di ordine n e non singolare:

$$\mathbf{Ax} = \mathbf{b} \implies \mathbf{USV}^T \mathbf{x} = \mathbf{b} \implies \mathbf{SV}^T \mathbf{x} = \mathbf{U}^T \mathbf{b} \implies \begin{cases} \mathbf{Sy} = \mathbf{U}^T \mathbf{b} \\ \mathbf{x} = \mathbf{Vy} \end{cases} \implies \mathbf{y}$$

Tuttavia il calcolo delle matrici $\mathbf{U}$, $\mathbf{V}$ ed $\mathbf{S}$ richiede $O(32n^3/3)$ operazioni aritmetiche mentre il metodo di Gauss ne richiede solamente $O(n^3/3)$.

2.6 Metodi iterativi

I metodi diretti non sono generalmente adeguati per sistemi di dimensioni elevate. Tali sistemi, che scaturiscono nella maggior parte dalle applicazioni, generalmente ammettono una matrice dei coefficienti **sparsa**. Una matrice di dimensioni $n \times n$ si ritiene sparsa quando il numero degli elementi diversi dallo zero è di ordine n .

La presenza di sparsità in una matrice rappresenta a priori un vantaggio, dal momento che memorizzando solo gli elementi non nulli, si possono ottenere notevoli risparmi di memoria e di operazioni.

MATLAB per esempio consente di memorizzare le matrici in un formato sparso, secondo cui si prendono in considerazione soltanto gli elementi non nulli.

In MATLAB

- `sparse(A)` consente di memorizzare solo gli elementi non nulli di $\mathbf{A}$;
- `spy(A)` consente di rappresentare graficamente l'insieme degli elementi non nulli di $\mathbf{A}$, ovvero il cosiddetto *pattern*;
- `nnz(A)` fornisce il numero degli elementi non nulli di $\mathbf{A}$;
- `spdiags(A,d,m,n)` crea una matrice sparsa $m \times n$, i cui elementi sulle diagonal, specificate dalle componenti del vettore $\mathbf{d}$, coincidono con gli elementi delle colonne di $\mathbf{A}$.

Gli standard di memorizzazione si chiamano CSR (Compressed Sparse Row Format) e COO (Coordinates Format).

Iniziamo dall'illustrare COO. Uso 3 vettori

$$\mathbf{r} = [\text{indici di riga dei non-zero}]$$

$\mathbf{c}$ = [indici di colonna dei non-zeri]

$\mathbf{v}$ = [valori dei non zeri]

Esempio 2.6.1.

$$\begin{pmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

$$\mathbf{r} = [1 \ 1 \ 3]$$

$$\mathbf{c} = [1 \ 3 \ 1]$$

$$\mathbf{v} = [1 \ 1 \ 1]$$

Il CSR è una variante del COO che comprime il vettore $\mathbf{r}$ degli indici di riga.

r_i = numero di non zeri precedenti alla riga $i + 1$

$\mathbf{c}$ = [indici di colonna]

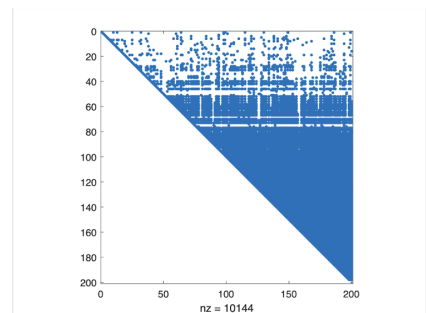
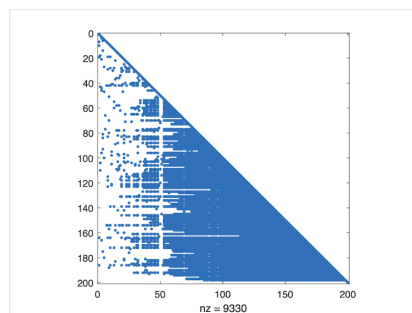
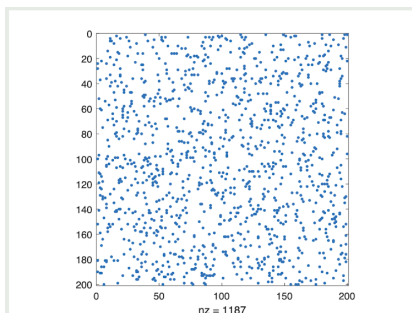
$\mathbf{v}$ = [valori]

Esempio 2.6.2. Con la stessa matrice dell'esempio precedente $\mathbf{r}$ diventa

$$\mathbf{r} = [2, 2]$$

Esempio 2.6.3. Sia $\mathbf{A}$ una matrice sparsa di ordine 200×200 , con un numero di elementi diversi da zero pari a $nz = 1187$.

Se si calcola la fattorizzazione $\mathbf{PA} = \mathbf{LU}$ della matrice $\mathbf{A}$, le matrici $\mathbf{L}$ e $\mathbf{U}$ non sono sparse a causa del fenomeno di fill-in; per entrambe si ha $nz \approx 10000$.



Per descrivere il generico metodo iterativo per la risoluzione di un sistema lineare, partiamo dal seguente sistema

$$\mathbf{Ax} = \mathbf{b} \iff \begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = b_n \end{cases}$$

e supponiamo che $a_{ii} \neq 0$ per ogni $i = 1, \dots, n$. Tale condizione, essendo $\mathbf{A}$ non singolare, si può realizzare mediante opportuni scambi di equazioni.

A partire da un arbitrario vettore iniziale $\mathbf{x}^{(0)}$, con un metodo iterativo si determina una sequenza di vettori

$$\{\mathbf{x}^{(k)}\}_{k=1,2,\dots}$$

che, sotto opportune condizioni, converge alla soluzione esatta $\mathbf{x}^*$ del sistema:

$$\mathbf{x}^{(0)} \implies \{\mathbf{x}^{(k)}\}_{k=1,2,\dots} : \lim_{k \rightarrow \infty} (\mathbf{x}^* - \mathbf{x}^{(k)}) = \mathbf{0} \quad (\text{vettore nullo})$$

2.6.1 Metodo di Jacobi e Gauss-Seidel

Definiamo dapprima due classici metodi iterativi. A tale scopo ricaviamo dall' i -esima equazione per $i = 1, \dots, n$ l'incognita x_i :

$$\begin{aligned} \sum_{j=1}^n a_{1j}x_j = b_1 &\implies a_{11}x_1 + \sum_{j=2}^n a_{1j}x_j = b_1 \\ \implies x_1 &= \frac{1}{a_{11}} \left(b_1 - \sum_{\substack{j=1 \\ j \neq 1}}^n a_{1j}x_j \right) \implies x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}x_j \right) \\ x_i &= \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j - \sum_{j=i+1}^n a_{ij}x_j}{a_{ii}} = F_i(\mathbf{x}), \quad a_{ii} \neq 0 \end{aligned}$$

e, a partire dal vettore $\mathbf{x}^{(0)}$, generiamo la successione di vettori

$$\{\mathbf{x}^{(k)}\}_{k=1,2,\dots}$$

la cui i -esima componente, per $i = 1, \dots, n$, è definita dall'espressione:

$$x_i^{(k)} = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k-1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)}}{a_{ii}} = F_i(\mathbf{x}^{(k-1)}), \quad k = 1, 2, \dots$$

Tale formula iterativa definisce il cosiddetto **metodo di Jacobi**.

Ponendo invece

$$x_i^{(k)} = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)}}{a_{ii}}, \quad k = 1, 2, \dots$$

otteniamo il cosiddetto **metodo di Gauss-Seidel**.

Osservazione: Nella prima sommatoria si prendono gli elementi di ordine (k) perché per $j < i$, ovvero $j = 1 : i - 1$, ho già calcolato $x_j^{(k)}$ e posso far convergere più velocemente il metodo.

Metodo Jacobi

Dato

$$\mathbf{x}^{(0)} \in \mathbb{R}^2$$

$$x_1^{(1)} = b_1 - a_{12}x_2^{(0)} \longrightarrow \text{calcolo } x_1^{(1)}$$

$$x_2^{(1)} = b_2 - a_{21}x_1^{(0)} \longrightarrow \text{non uso } x_1^{(1)}$$

Metodo di Gauss-Seidel

$$x_1^{(1)} = b_1 - a_{12}x_2^{(0)} \longrightarrow \text{calcolo } x_1^{(1)}$$

$$x_2^{(1)} = b_2 - a_{21}x_1^{(1)} \longrightarrow \text{uso } x_1^{(1)}$$

Esempio 2.6.4.

$$\begin{cases} 3x_1 - x_2 = 2 \\ x_1 + 2x_2 = 3 \end{cases} \implies \begin{cases} x_1 = \frac{2+x_2}{3} \\ x_2 = \frac{3-x_1}{2} \end{cases} \quad \mathbf{x}^* = (1, 1)^T$$

METODO DI JACOBI:

$$\begin{cases} x_1^{(k)} = \frac{2+x_2^{(k-1)}}{3} \\ x_2^{(k)} = \frac{3-x_1^{(k-1)}}{2} \end{cases} \quad \mathbf{x}^{(0)} = (0, 0)^T$$

$\mathbf{x}^{(0)}$	$\mathbf{x}^{(1)}$	$\mathbf{x}^{(2)}$	$\mathbf{x}^{(3)}$	$\mathbf{x}^{(4)}$
0	$\frac{2}{3}$	$\frac{7}{6}$	$\frac{19}{18}$	$\dots$
0	$\frac{3}{2}$	$\frac{7}{6}$	$\frac{11}{12}$	$\dots$

$$\frac{\|\mathbf{x}^* - \mathbf{x}^{(3)}\|_\infty}{\|\mathbf{x}^*\|_\infty} \approx 0.8 \cdot 10^{-1}$$

METODO DI GAUSS-SEIDEL:

$$\begin{cases} x_1^{(k)} = \frac{2+x_2^{(k-1)}}{3} \\ x_2^{(k)} = \frac{3-x_1^{(k)}}{2} \end{cases} \quad \mathbf{x}^{(0)} = (0, 0)^T$$

$\mathbf{x}^{(0)}$	$\mathbf{x}^{(1)}$	$\mathbf{x}^{(2)}$	$\mathbf{x}^{(3)}$	$\mathbf{x}^{(4)}$
0	$\frac{2}{3}$	$\frac{19}{18}$	$\frac{107}{108}$	$\dots$
0	$\frac{7}{6}$	$\frac{35}{36}$	$\frac{217}{216}$	$\dots$

$$\frac{\|\mathbf{x}^* - \mathbf{x}^{(3)}\|_\infty}{\|\mathbf{x}^*\|_\infty} \approx 0.9 \cdot 10^{-2}$$

Quando si implementa un procedimento iterativo occorre definire dei criteri d'arresto.

Un possibile criterio d'arresto consiste senz'altro nel fissare un numero massimo $kmax$ di iterazioni.

Un altro criterio d'arresto riguarda la bontà dell'approssimazione $\mathbf{x}^{(k)}$ e consiste nel fissare una tolleranza relativa $tol_r (\geq \varepsilon_m)$ o assoluta tol_a e nell'arrestare il processo all'iterazione $\bar{k}$ se $\mathbf{x}^{(\bar{k})}$ soddisfa la seguente disuguaglianza

$$\left\| \mathbf{x}^{(\bar{k}+1)} - \mathbf{x}^{(\bar{k})} \right\| \leq tol_r \left\| \mathbf{x}^{(\bar{k}+1)} \right\|$$

oppure

$$\left\| \mathbf{x}^{(\bar{k}+1)} - \mathbf{x}^{(\bar{k})} \right\| \leq tol_a$$

Osservazione: Sto verificando idealmente che la successione sia di Cauchy perché $\mathbb{R}^n$ è completo.

Osservazione: Osserviamo che tale criterio d'arresto ha senso quando il metodo converge “rapidamente”.

Seguono implementazioni in MATLAB dei metodi di Jacobi e Gauss-Seidel con iterazioni massime e controllo dell'errore implementati.

```
1 function [x,k,ier] = Jacobi(A,b,x0,tol,kmax)
2 if prod(diag(A)) == 0
3     x = []; k = []; ier = -1;
4     return
5 end
6 D = diag(diag(A));
7 C = A-D;
8 k = 0; err = Inf;
9 while k <= kmax && err > tol
10     x = D\b-C*x0;
11     err = norm(x-x0)/norm(x);
12     k = k + 1;
13     x0 = x;
14 end
15 if k == kmax+1 && err > tol
16     ier = 0;
17 else
18     ier = 1;
19 end
```

```
1 function [x,k,ier] = Gauss_Seidel(A,b,x0,tol,kmax)
2 if prod(diag(A)) == 0
3     x = []; k = []; ier = -1;
4     return
5 end
6 D = tril(A);
7 C = A-D;
8 k = 0; err = Inf;
9 while k <= kmax && err > tol
10     x = D\b-C*x0;
11     err = norm(x-x0)/norm(x);
12     k = k + 1;
13     x0 = x;
```

```

14 end
15 if k == kmax+1 && err > tol
16     ier = 0;
17 else
18     ier = 1;
19 end

```

2.6.2 Generalizzazione di Jacobi e Gauss-Seidel

I metodi di Jacobi e di Gauss-Seidel fanno parte di quella classe più generale di procedimenti iterativi che si deducono nel seguente modo:

$$\mathbf{Ax} = \mathbf{b} \iff (\mathbf{D} + \mathbf{C})\mathbf{x} = \mathbf{b} \iff \mathbf{D}\mathbf{x} = -\mathbf{C}\mathbf{x} + \mathbf{b}$$

$$\implies \mathbf{D}\mathbf{x}^{(k)} = -\mathbf{C}\mathbf{x}^{(k-1)} + \mathbf{b}, \quad k = 1, 2, \dots$$

ove, a ogni iterazione k , $\mathbf{x}^{(k)}$ è la soluzione di un sistema lineare con matrice dei coefficienti $\mathbf{D}$ e termine noto $\mathbf{b} - \mathbf{C}\mathbf{x}^{(k-1)}$.

Pertanto la matrice $\mathbf{D}$ deve essere:

1. non singolare, ovvero tale da garantire l'esistenza e l'unicità della soluzione $\mathbf{x}^{(k)}$ a ogni passo k ;
2. di forma semplice (triangolare, diagonale), ovvero tale che la soluzione $\mathbf{x}^{(k)}$ del sistema possa ottenersi, a ogni passo k , con un algoritmo relativamente semplice e poco costoso;
3. tale da garantire la convergenza, per $k \rightarrow \infty$, di $\mathbf{x}^{(k)}$ alla soluzione $\mathbf{x}^*$ del sistema originario.

Osserviamo che, nel caso del metodo di Jacobi

$$a_{ii}x_i^{(k)} = b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k-1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)}, \quad i = 1, \dots, n,$$

le matrici $\mathbf{D}$ e $\mathbf{C}$ dello sdoppiamento $\mathbf{A} = \mathbf{D} + \mathbf{C}$ sono così definite

$$\mathbf{D} = \begin{pmatrix} a_{11} & 0 & \dots & 0 \\ 0 & a_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{nn} \end{pmatrix}, \quad \mathbf{C} = \mathbf{A} - \mathbf{D};$$

nel caso del metodo di Gauss-Seidel

$$\sum_{j=1}^i a_{ij}x_j^{(k)} = b_i - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)}, \quad i = 1, \dots, n$$

si ha

$$\mathbf{D} = \begin{pmatrix} a_{11} & 0 & \dots & 0 \\ a_{21} & a_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix}, \quad \mathbf{C} = \mathbf{A} - \mathbf{D}.$$

2.6.3 Convergenza di metetodi iterativi

Per lo studio della convergenza dei metodi iterativi, definiamo il vettore errore

$$\mathbf{e}^{(k)} := \mathbf{x}^* - \mathbf{x}^{(k)}$$

e sottraiamo membro a membro le seguenti due equazioni:

$$\begin{aligned}\mathbf{D}\mathbf{x}^* &= \mathbf{b} - \mathbf{C}\mathbf{x}^* \\ \mathbf{D}\mathbf{x}^{(k)} &= \mathbf{b} - \mathbf{C}\mathbf{x}^{(k-1)} \\ \mathbf{D}\mathbf{e}^{(k)} &= -\mathbf{C}\mathbf{e}^{(k-1)}\end{aligned}$$

Abbiamo pertanto

$$\mathbf{e}^{(k)} = -\mathbf{D}^{-1}\mathbf{C}\mathbf{e}^{(k-1)} = \mathbf{B}\mathbf{e}^{(k-1)}$$

ove abbiamo posto

$$\mathbf{B} := -\mathbf{D}^{-1}\mathbf{C} = -\mathbf{D}^{-1}(\mathbf{A} - \mathbf{D}) = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$$

La matrice $\mathbf{B}$ è detta **matrice di iterazione**.

Tenendo conto che l'uguaglianza

$$\mathbf{e}^{(k)} = \mathbf{B}\mathbf{e}^{(k-1)}$$

vale per ogni $k = 1, 2, \dots$, si ha

$$\mathbf{e}^{(k)} = \mathbf{B}\mathbf{e}^{(k-1)} = \mathbf{B}^2\mathbf{e}^{(k-2)} = \dots = \mathbf{B}^k\mathbf{e}^{(0)}$$

da cui deduciamo che

$$\lim_{k \rightarrow \infty} \mathbf{e}^{(k)} = \mathbf{0} \quad (\text{vettore nullo}) \quad \forall \mathbf{x}^{(0)} \iff \lim_{k \rightarrow \infty} \mathbf{B}^k = \mathbf{0} \quad (\text{matrice nulla})$$

Ricordando che

$$\mathbf{B} = \mathbf{V}^{-1}\mathbf{D}\mathbf{V} \implies \mathbf{B}^2 = \mathbf{V}^{-1}\mathbf{D}\mathbf{V}\mathbf{V}^{-1}\mathbf{D}\mathbf{V} = \mathbf{V}^{-1}\mathbf{D}^2\mathbf{V} \implies \mathbf{B}^k = \mathbf{V}^{-1}\mathbf{D}^k\mathbf{V}$$

ovvero che gli autovalori di $\mathbf{B}^k$ sono λ_i^k con λ_i autovalori di $\mathbf{B}$ e notando che l'unica matrice con autovalori tutti nulli è la matrice nulla si può dimostrare che

$$\lim_{k \rightarrow \infty} \mathbf{B}^k = \mathbf{0} \iff \rho(\mathbf{B}) < 1$$

ove $\rho(\mathbf{B})$ è il raggio spettrale della matrice, cioè

$$\rho(\mathbf{B}) = \max_{1 \leq i \leq n} |\lambda_i|$$

con λ_i autovalore di $\mathbf{B}$, allora possiamo dedurre

$$\lim_{k \rightarrow \infty} \mathbf{e}^{(k)} = \mathbf{0} \quad \forall \mathbf{x}^{(0)} \iff \rho(\mathbf{B}) < 1$$

Ricordando che per le norme $1, 2, \infty$ sussiste la proprietà

$$\rho(\mathbf{A}) \leq \|\mathbf{A}\|,$$

possiamo senz'altro affermare che, se per almeno una delle suddette norme vale

$$\|\mathbf{B}\| < 1,$$

allora

$$\lim_{k \rightarrow \infty} \mathbf{e}^{(k)} = \mathbf{0} \quad \forall \mathbf{x}^{(0)}.$$

Convergenza dei metodi di Jacobi e Gauss-Seidel

Teorema 2.6.5. Se $\mathbf{A}$ è a diagonale dominante per righe o per colonne, allora i metodi di Jacobi e di Gauss-Seidel, applicati al sistema lineare $\mathbf{Ax} = \mathbf{b}$, convergono qualunque sia il vettore iniziale.

Teorema 2.6.6. Se $\mathbf{A}$ è simmetrica e definita positiva, allora il metodo di Gauss-Seidel converge.

Osservazione:

1. La convergenza del metodo di Gauss-Seidel non implica la convergenza del metodo di Jacobi e viceversa. Se entrambi convergono, in generale il metodo di Gauss-Seidel converge più rapidamente.
2. La rapidità di convergenza del metodo iterativo

$$\mathbf{D}\mathbf{x}^{(k)} = -\mathbf{C}\mathbf{x}^{(k-1)} + \mathbf{b}, \quad k = 1, 2, \dots$$

dipende dal raggio spettrale $\rho(\mathbf{B})$ della matrice di iterazione

$$\mathbf{B} = -\mathbf{D}^{-1}\mathbf{C} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A} :$$

quanto più esso è piccolo, tanto più è rapida la convergenza del corrispondente metodo.

Teorema 2.6.7. Se $\mathbf{A}$ è tridiagonale, non singolare con elementi diagonali non nulli, allora i metodi di Jacobi e di Gauss-Seidel sono entrambi convergenti o entrambi divergenti. Nel caso di convergenza, il metodo di Gauss-Seidel converge più velocemente di quello di Jacobi e, in particolare, si ha

$$\rho(\mathbf{B}_{GS}) = \rho(\mathbf{B}_J)^2.$$

2.6.4 Rilassamento del metodo di Gauss-Seidel

Definiamo ora un'altra classe di metodi iterativi detti metodi di rilassamento. A partire dalla formula che definisce il metodo di Gauss-Seidel

$$x_i^{(k)} = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)}}{a_{ii}}, \quad k = 1, 2, \dots$$

sottraiamo $x_i^{(k-1)}$, per $i = 1, 2, \dots, n$, da ambo i membri.

$$\begin{aligned} x_i^{(k)} - x_i^{(k-1)} &= \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)} \right) - x_i^{(k-1)} \\ &= \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)} - a_{ii}x_i^{(k-1)} \right) \\ &= \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i}^n a_{ij}x_j^{(k-1)} \right) \end{aligned}$$

ottenendo quindi

$$r_i^{(k-1)} = x_i^{(k)} - x_i^{(k-1)} = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i}^n a_{ij}x_j^{(k-1)}}{a_{ii}}, \quad k = 1, 2, \dots$$

dove $r_i^{(k-1)}$ rappresenta la correzione da apportare a $x_i^{(k-1)}$ per ottenere $x_i^{(k)}$:

$$x_i^{(k)} = x_i^{(k-1)} + r_i^{(k-1)}, \quad k = 1, 2, \dots, \quad i = 1, \dots, n$$

Quest'ultima relazione definisce ancora il metodo di Gauss-Seidel. Essa tuttavia suggerisce l'introduzione di un parametro $\omega \neq 0$ al fine di migliorare la correzione su $x_i^{(k-1)}$:

$$x_i^{(k)} = x_i^{(k-1)} + \omega r_i^{(k-1)}, \quad k = 1, 2, \dots, \quad i = 1, \dots, n$$

Tale relazione definisce il **metodo di rilassamento**: il metodo è detto di **sottorilassamento** se $\omega < 1$, mentre è detto di **sovrarilassamento** se $\omega > 1$ (se $\omega = 1$ si ha il metodo di Gauss-Seidel). Quest'ultimo è detto metodo **SOR**, dalle iniziali di Successive Over Relaxation.

Per riscrivere il metodo in forma matriciale, introduciamo le seguenti matrici che, per comodità, definiamo con i seguenti comandi MATLAB:

$$\mathbf{D} = \text{diag}(\text{diag}(\mathbf{A})), \quad \mathbf{L} = \text{tril}(\mathbf{A}, -1), \quad \mathbf{U} = \text{triu}(\mathbf{A}, 1).$$

Si ha pertanto $\mathbf{A} = \mathbf{L} + \mathbf{D} + \mathbf{U}$ e riscrivendo in forma matriciale le iterazioni di Gauss-Seidel

$$(\mathbf{D} + \mathbf{L})\mathbf{x}^{(k)} = \mathbf{b} - \mathbf{U}\mathbf{x}^{(k-1)}$$

$$\mathbf{D}\mathbf{x}^{(k)} = \mathbf{b} - \mathbf{L}\mathbf{x}^{(k)} - \mathbf{U}\mathbf{x}^{(k-1)}$$

Sottraendo $\mathbf{D}\mathbf{x}^{(k-1)}$ da ambo i membri, si ha

$$\underbrace{\mathbf{D}\mathbf{x}^{(k)} - \mathbf{D}\mathbf{x}^{(k-1)}}_{\mathbf{D}\mathbf{r}^{(k-1)}} = \mathbf{b} - \mathbf{L}\mathbf{x}^{(k)} - (\mathbf{D} + \mathbf{U})\mathbf{x}^{(k-1)}$$

ovvero

$$\mathbf{r}^{(k-1)} = \mathbf{D}^{-1} [\mathbf{b} - \mathbf{L}\mathbf{x}^{(k)} - (\mathbf{D} + \mathbf{U})\mathbf{x}^{(k-1)}]$$

da cui rilassando si ottiene

$$\begin{aligned} \mathbf{x}^{(k)} &= \mathbf{x}^{(k-1)} + \omega \mathbf{r}^{(k-1)} \\ &= \mathbf{x}^{(k-1)} + \omega \mathbf{D}^{-1} [\mathbf{b} - \mathbf{L}\mathbf{x}^{(k)} - (\mathbf{D} + \mathbf{U})\mathbf{x}^{(k-1)}] \end{aligned}$$

In definitiva, abbiamo

$$(\mathbf{D} + \omega \mathbf{L})\mathbf{x}^{(k)} = \omega \mathbf{b} + [(1 - \omega)\mathbf{D} - \omega \mathbf{U}]\mathbf{x}^{(k-1)}$$

Ricordando il risultato di convergenza dei metodi iterativi, abbiamo che la matrice di iterazione è data da:

$$\mathbf{B} = (\mathbf{D} + \omega \mathbf{L})^{-1} [(1 - \omega)\mathbf{D} - \omega \mathbf{U}]$$

e il metodo di rilassamento risulterà convergente se e solo se $\rho(\mathbf{B}) < 1$. Notiamo che in Gauss-Seidel la matrice di iterazione trovata era

$$\mathbf{B} = -(\mathbf{D} + \mathbf{L})^{-1}$$

che corrisponde di nuovo al caso $\omega = 1$.

La scelta del parametro ω deve rendere $\rho(\mathbf{B})$ quanto più piccolo è possibile. In alcuni casi è possibile determinare il valore di ω_{opt} per il quale $\rho(\mathbf{B})$ assume valore minimo.

Si dimostra che $\rho(\mathbf{B}) \geq 1$ quando $\omega \leq 0$ oppure $\omega \geq 2$ e, pertanto, per questi valori di ω , il metodo non può convergere.

La **convergenza** è invece **assicurata** per tutti i valori di $0 < \omega < 2$ quando $\mathbf{A}$ è simmetrica e definita positiva.

Segue un'implementazione in MATLAB del SOR.

```

1 function [x,k,ier] = SOR(A,b,x0,omega,tol,kmax)
2 if prod(diag(A)) == 0
3     x = []; k = []; ier = -1;
4     return
5 end
6 D = diag(diag(A));
7 L = tril(A,-1);
8 U = triu(A,1);
9 k = 0; err = Inf;
10 while k <= kmax && err > tol
11     x = (D+omega*L)\(omega*b+((1-omega)*D-omega*U)*x0);
12     err = norm(x-x0)/norm(x);
13     k = k + 1;
14     x0 = x;
15 end
16 if k == kmax+1 && err > tol
17     ier = 0;
18 else
19     ier = 1;
20 end

```

2.6.5 Metodi di discesa